

Linear Programming of Graph Squares

Eddie Xu

Submitted for the degree of Master of Science:
Mathematics and Statistics

Supervisor: Christopher Duffy

September 30, 2026

Abstract

The square of a graph is obtained by joining every pair of vertices whose distance is at most two. Given a graph G , the graph square root problem asks whether there exists a graph H such that $G = H^2$. Whilst this problem admits polynomial-time algorithms for restricted classes of root graphs, the general decision problem is **NP**-complete.

In this thesis, rather than studying the construction problem directly and using reduction techniques to ascertain polynomial-time algorithms, we formulate the problem as an integer program based on the relationship between the adjacency matrices of a root H and its square G . By constructing the corresponding linear relaxation, the problem can then be situated within polynomial time through the use of interior-point methods.

The seminal result for the problem is the work by Ross and Harary, who approached the problem by restricting the root graphs to trees. By analyzing the number of cliques which each vertex in the graph square is situated in, they gave an algorithm for finding the unique tree root up to isomorphism. In contrast, our formulation provides an optimization-based approach that can be iteratively strengthened through the addition of further constraints. These constraints refine the linear relaxation, enabling tree roots to be represented by integral solutions to the linear program, while graphs with no possible roots are identified through fractional or infeasible solutions. Rather than relying on heuristic such as rounding, our approach exploits structural properties of the graph square and its root to obtain an exact solution.

Contents

1	Introduction	2
1.1	Background	2
1.2	Literature	3
1.2.1	Overview	4
1.3	Definitions and Notations	5
1.3.1	Complexity Theory	5
1.3.2	Graph Theory	7
1.3.3	Mathematical Optimization	10
2	Algorithmic Approaches	12
2.1	NP-Completeness Proof	12
2.1.1	NP-Hard proof	13
2.2	Algorithm for Tree Roots	17
2.3	Mathematical Optimization Approach	21
2.3.1	Formulation	23
2.3.2	Validity	25
3	Square of a Tree using Linear Programming	28
3.1	Linear Program Relaxation	28
3.2	Min Linear Program	30
3.3	Max Linear Program	31
3.3.1	Fundamental Properties	32
3.4	Leaf Fixing	33
3.4.1	Single Triangular Decomposition	34
3.4.2	Tetrahedron Decomposition	35
3.5	Unicliqual Vertex Linear Program	36
3.6	Tree Proof	37
4	Further Work	41
4.1	Vertex Deletion	41
4.2	Symbolic Groups	45
4.3	Duality	46
4.4	Rounding	48

Chapter 1

Introduction

1.1 Background

In 1949, Leon Festinger proposed a novel method in the field of social psychology for analyzing relationships among a number of people present [FBB23]. Classically, this was represented using a sociogram, a visual tool that maps out the relationships between people. However, Forsyth and Katz introduced the use of a matrix to represent the sociometric pattern, with rows and columns corresponding to individuals and containing 1s for choices and blank diagonals. Festinger noticed that his structure, reminiscent of the adjacency matrix defined by König in [Kön36], also had the remarkable property that the action of squaring a matrix

$$A_{rc}^2 = A_{1c}A_{r1} + \cdots + A_{nc}A_{rn}^1$$

represented the number of “two-step” connections, which can then be used to denote “three-step” connections between any two people

$$A_{rc}^3 = A_{1c}A_{r1}^2 + \cdots + A_{nc}A_{rn}^2,^2$$

such that higher steps can be easily generated. In this manner, higher powers of the adjacency matrix encode the number of walks of increasing length between pairs of individuals, allowing densely connected groups of people, known as cliques, to be identified.

In the process of improving Festinger’s clique detection methods, Ross and Harary showed that complete graphs may be realized as squares of stars by considering powers of such matrices [RH60]. They introduced a structural characterization of the square of a graph in terms of clique intersections, “multicliquical” vertices, and so called “neighborly” properties, proving that a graph is the square of a tree if and only if

- i) every vertex is every vertex is adjacent to at least one other vertex,
- ii) no two maximal cliques intersect in more than two vertices, and

¹ A_{rc}^2 is more naturally written as $\sum_{k=1}^n A_{ck}A_{kr}$ using the commutativity of multiplication $A_{1c}A_{r1} = A_{c1}A_{1r}$, which intuitively gives the notion of paths from c to r via intermediary k

² A_{rc}^3 is more naturally written as $\sum_{k=1}^n A_{ck}A_{kr}^2$, which intuitively counts the number of paths from c to r that go through an intermediate vertex k , followed by any two-step path from k to r .

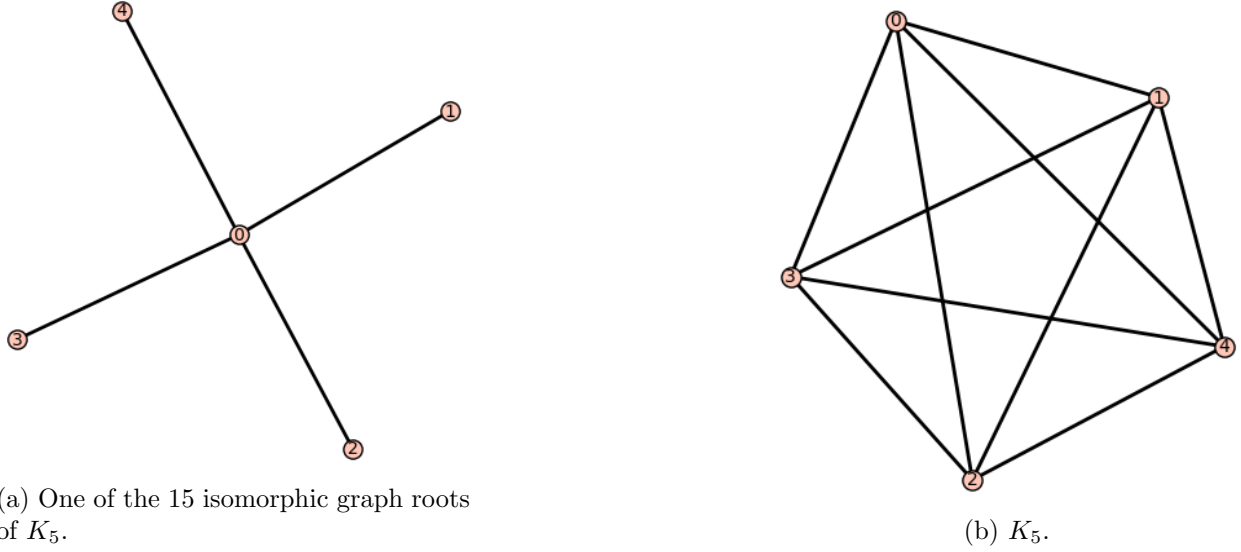


Figure 1.1: The star graph $K_{1,4}$ and its square K_5 , which is a complete graph since every vertex in the root .

- iii) the number of maximal clique pairs intersecting in two vertices is exactly one less than the total number of maximal cliques.

Using these properties, they constructed an explicit algorithm that recovers the underlying tree for a given graph square. Their work has become the foundational reference for the study of graph squares and is universally cited in subsequent literature on the subject, and will be explored in more detail later.

Since then, the problem of finding the square root of a graph has extended beyond sociology and is a well-studied field in both mathematics and computer-science, with direct applications in distributed computing (e.g., see [PA23]), network design and computational biology (e.g., see [Zho+25]). This is in part because many structural properties of graph roots are first established in the square case before being extended to the more general k -th power setting (e.g., see [HKT67]).

Research in this area has primarily focused around an undirected graph H and its square $G = H^2$ which is obtained by adding an edge between every pair of vertices whose distance in H is at most two. This two-step undirected graph square has been studied extensively with much of the focus being on the complexity of the decision problem - given a specific graph square G does there exist a root graph H such that $H^2 = G$. Motwani and Sudan showed that deciding if a graph admits a root is **NP**-complete. Subsequent studies in the area have restricted the decision problem in two ways, either by considering various properties on the root graph or by considering properties on the graph square.

1.2 Literature

Focusing first on special undirected graph classes, the aforementioned work by Ross and Harary in 1960 gives an algorithm for constructing a tree root by using characterizations of the graph square. As part of this work they proved tree square roots are unique up to isomorphism. This was the first major contribution to the study of graph roots and provided fundamental structural characteristics of graph squares, laying the groundwork for later research on graph root problems and their

applications. Building upon these insights, Lin and Skiena bypassed the exponential bottleneck of general maximal clique enumeration present in Ross and Harary’s approach, by developing an optimal $\mathcal{O}(|V| + |E|)$ linear time algorithm to recognize tree squares and reconstruct their unique tree roots [LS95].

Fleischner in 1974 showed that as long as a graph is 2-connected its graph square will have a Hamiltonian cycle. This established a broad condition guaranteeing Hamiltonicity in graph squares which is used in many subsequent works such as [BMR14].

Lau and Corneil in 2004 provided an algorithm for constructing square roots from an interval square in polynomial time, and showed that the decision problem for finding roots of chordal graphs, roots of split graphs, and chordal roots of graphs are **NP**-complete [LC04]. Furthermore, Lau gave a linear-time algorithm for constructing square roots of bipartite graphs, and proved that deciding whether a bipartite graph has a cube root is **NP**-complete [Lau06]. Le and Tuy in 2007 generalized tree-root results further by giving a linear time recognition of roots of block graphs [LT07].

Generalizing the notion of graph squares, in 1967 Mukhopadhyay provided necessary and sufficient conditions for a connected graph to have a square root [Muk67], whilst Geller in 1970 likewise found necessary and sufficient conditions for a digraph to have a square root [Gel70].

In 1994 Motwani and Sudan established the complexity of the square problem for the generalized undirected case and showed that the problem of determining if a given graph has a square root is an **NP**-complete problem via a polynomial reduction to 3-CNF not-all-equal SAT [MS94].

Their work led to joint work by Farzad, Le, Tuy, Lau and Karimi in 2009 and 2012, in which they established a partial dichotomy theorem for the complexity of determining if a given undirected graph of girth g has a square root [Far+09] which was informally completed under [FK12]. In 2013, Cochefert, Couturier, Golovach, Kratsch, and Paulusma gave an algorithm for constructing a square root with minimal number of edges from undirected graphs with max degree ≤ 6 in polynomial time and an algorithm for constructing maximal edge square roots in exponential time [Coc+13].

In recent years, Duffy provided a full classification for the undirected square of oriented graphs satisfying max degree ≤ 3 and girth ≥ 4 [Duf25].

1.2.1 Overview

In addition to providing a general overview of the research direction in this section we first standardise the definitions that will be used for the rest of this thesis. We then introduce fundamental theory on the three areas of: complexity theory, graph theory and mathematical optimization that are used in the various algorithmic approaches to the decision problem.

The first construction we examine is the reduction used by Motwani and Sudan in showing that the decision problem determining whether a given graph G admits a square root is **NP**-complete [MS94]. Next, we introduce the polynomial time algorithm given by Ross and Harary for constructing a tree root from a given square in detail. Finally, we formulate the problem as an integer program.

In Chapter 3 we present the ideas of linear relaxation and discuss the results of each program and their validity. To obtain these results, we conducted an exhaustive enumeration on all non-isomorphic connected graphs up to $n = 8$ vertices. For each such graph, we formulated the corresponding linear program and solved it using a standard LP solver using interior point method. The outcomes of these computations are categorized as integral, fractional, or infeasible and are analyzed.

Finally, we introduce several augmentations to the linear program, consisting of constraints that encode structural bounds on both the graph square and its underlying root. In particular, we show that by adding specific constraints to the LP, we can ensure that the square of all trees will

return a feasible integer solution. This augmentation ensures that the LP returns a tree root in polynomial time, whilst ensuring that all squares with roots remain LP feasible.

1.3 Definitions and Notations

1.3.1 Complexity Theory

Complexity theory studies the classification of computational problems and explores the amount of resources required to solve these problems. Complexity Theory considers both time and space as critical resources and is standardized across various different problems.

Definition 1.3.1 (Big- O Notation). *Let $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$. The set $O(g(n))$ is defined as*

$$O(g(n)) = \{f \mid \exists c > 0, n_0 \in \mathbb{N} \text{ such that } f(n) \leq cg(n) \text{ for all } n \geq n_0\}.$$

We write $f(n) = O(g(n))$ as shorthand for $f(n) \in O(g(n))$.

Using this notation, we can classify algorithms based on their runtime.

Definition 1.3.2 (Polynomial-Time Algorithm). *An algorithm is said to run in **polynomial time** if there exists a constant $c \geq 0$ such that its run time on inputs of size n is $O(n^c)$.*

Polynomial-time algorithms are important because they are generally regarded as efficient and computationally “simple”. Consequently, the class of problems that admit polynomial-time algorithms forms the basis for the complexity class **P**.

Definition 1.3.3 (Decision Problem). *A **decision problem** is a computational problem whose output is either **Yes** or **No** for every valid input instance.*

Definition 1.3.4 (Class **P**). *The complexity class **P** consists of all decision problems that can be solved by a deterministic algorithm in polynomial time.*

Unfortunately many important decision problems are not known to admit such polynomial-time algorithms and hence need additional classification on their “hardness”. For example, determining whether a graph is 3-colourable is a well-known problem for which no polynomial-time algorithm is known, despite the fact that a proposed 3-colouring can be checked efficiently. This motivates the study of complexity classes based not only on how efficiently a solution can be found, but also on how a proposed solution can be verified efficiently.

Definition 1.3.5 (Certificate). *Let L be a decision problem. A **certificate** for an instance x of L is a finite string c that provides evidence that x is a **Yes**-instance of L . A certificate is said to be **polynomially verifiable** if there exists a deterministic algorithm that for given input (x, c) , can determine whether c is a valid certificate for x in time polynomial in $|x|$.*

Definition 1.3.6 (Class **NP**). *The complexity class **NP** consists of all decision problems for which every **Yes**-instance admits a certificate that is polynomially verifiable.*

Since every decision problem in **P** can be solved in polynomial time, a solution can also be verified in polynomial time,

$$\mathbf{P} \subseteq \mathbf{NP}.$$

Whether this inclusion is strict remains one of the most important open questions in theoretical computer science - the **P** vs **NP** problem. As **NP** is a large class of decision problems, the relative difficulty of problems in **NP** is compared by using reduction.

Definition 1.3.7 (Polynomial-Time Reduction). Let A and B be decision problems. A **polynomial-time reduction** from A to B , denoted $A \leq_p B$, is a polynomial-time computable function f such that, for every instance x of A ,

$$x \in A \quad \text{if and only if} \quad f(x) \in B.$$

This notion of reduction allows the computational difficulty of problems to be compared. In particular, a problem is **NP-hard** if every problem in **NP** can be reduced to it via a polynomial time reduction.

Definition 1.3.8 (Class NP-Hard). A decision problem B is **NP-hard** if every problem $A \in \mathbf{NP}$ satisfies $A \leq_p B$. The collection of all **NP-hard** problems is denoted by the class **NP-hard**.

Importantly, an **NP-hard** problem may not necessarily lie in **NP**. If it does, it is called **NP-complete**.

Definition 1.3.9 (Class NP-Complete). A decision problem B is **NP-complete** if $B \in \mathbf{NP}$, and B is **NP-hard**. The collection of all **NP-complete** problems is denoted by the class **NP-complete**.

One can establish that a problem is **NP-complete** by showing that it belongs to **NP** and that a known **NP-complete** problem can be reduced to it in polynomial time.

The first known **NP-complete** problem was established by the Cook-Levin theorem and is the Boolean satisfiability problem, or SAT.

Definition 1.3.10 (Boolean Satisfiability Problem). The **Boolean satisfiability problem (SAT)** asks whether there exists an assignment of truth values to the variables of a given Boolean formula such that the formula evaluates to true. A Boolean formula is said to be **satisfiable** if such an assignment exists.

Any Boolean formula can be transformed into an equisatisfiable formula typically in linear time using the **Tseytin transformation** by introducing additional variables. This transformation preserves satisfiability while ensuring only linear growth in the size of the resulting conjunctive normal form Boolean formula.

Definition 1.3.11 (Conjunctive Normal Form). A Boolean formula is in **conjunctive normal form (CNF)** if it is expressed as a conjunction of clauses, where each clause is a disjunction of literals. That is, a CNF formula has the form

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m, \quad \text{where each clause } C_i \text{ is of the form } C_i = \ell_{i1} \vee \ell_{i2} \vee \cdots \vee \ell_{ik_i}$$

and each ℓ_{ij} is either a Boolean variable or its complement, denoted with $\bar{\ell}_{ij}$.

A further restriction is obtained by requiring each clause in a CNF formula to contain exactly three literals, which can be done using 3-CNF transformation, to give the following problem.

Definition 1.3.12 (3-SAT). The **3-SAT** problem asks whether there exists an assignment of truth values to the variables of a Boolean formula in conjunctive normal form such that every clause contains exactly three literals and the formula evaluates to true. It is formally structured as

Instance: A Boolean formula φ in conjunctive normal form in which each clause contains exactly three literals.

Question: Does there exist an assignment of truth values to the variables of φ such that every clause is satisfied?

The decision problem **3-SAT** is a well-known **NP-complete** problem. It is commonly used as a starting point for proving the **NP-completeness** of other decision problems. In particular, we will reduce 3-SAT to the **SQUARE** problem to establish that the latter is **NP-complete**.

1.3.2 Graph Theory

Informally, a graph is relational structure that captures relationships between discrete objects.

Definition 1.3.13 (Graph). A **graph** G is an ordered pair (V, E) , where V is a set of vertices and E is a set of unordered pairs of distinct vertices in V . Each element of E is called an **edge** of G .

The number of vertices and edges in a graph provide basic measures of its order and size.

Definition 1.3.14 (Order and Size). The **order** of a graph G , denoted by $|G|$, is the number of vertices in G , while the **size** of G is the number of edges. Thus, if $V_G = \{v_1, \dots, v_n\}$ and $E_G = \{e_1, \dots, e_m\}$, then $|G| = n$ and $|E_G| = m$.

For a vertex v , the number of edges incident to it is its degree.

Definition 1.3.15 (Degree). The **degree** of a vertex v in a graph G , denoted by $\deg_G(v)$, is the number of edges incident to v . The minimum and maximum degrees of G are denoted by

$$\delta(G) = \min_{v \in V_G} \deg_G(v) \quad \text{and} \quad \Delta(G) = \max_{v \in V_G} \deg_G(v),$$

respectively.

The degree of a vertex can also be described through the vertices to which it is adjacent to.

Definition 1.3.16 (Neighbourhood). For a vertex $v \in V_G$, the **open neighbourhood** of v denoted by $N_G(v)$, is the set of vertices adjacent to v in G . The **closed neighbourhood** of v includes vertex v and is denoted by

$$N_G[v] = N_G(v) \cup \{v\}.$$

Connections between vertices in a graph can be described using walks, paths and cycles.

Definition 1.3.17 (Walk). A **walk** from a vertex v_i to a vertex v_j in a graph G is an alternating sequence of vertices and edges

$$W = \langle v_i, e_1, v_1, e_2, \dots, e_k, v_j \rangle,$$

where each edge e_r has endpoints given by the vertices immediately preceding and following it in the sequence. The number of edges in W is called the **length** of the walk.

Definition 1.3.18 (Path). A **path** is a walk in which no vertex is repeated. A path containing p vertices is denoted by P_p .

Definition 1.3.19 (Cycle). A **cycle** is a closed path in which the first and last vertices are the same and all other vertices are distinct. A cycle containing c vertices is denoted by C_c .

The length of a shortest cycle is an important structural parameter of a graph.

Definition 1.3.20 (Girth). The **girth** of a graph G , denoted by $g(G)$, is the length of the shortest cycle in G . If G contains no cycle, its girth is ∞ .

A connected graph with infinite girth is called a tree.

Definition 1.3.21 (Tree). A **tree** is a connected graph that contains no cycles.

The vertices of a tree can be classified according to their degree.

Definition 1.3.22 (Leaf). A **leaf** of a tree T is a vertex of degree one.

Definition 1.3.23 (Stem). A **stem** of a tree T is the vertex adjacent to a leaf of T .

The length of a shortest path provides a natural measure of the distance between two vertices.

Definition 1.3.24 (Distance). The **distance** between two vertices u and v on the graph G , denoted by $d_G(u, v)$, is the number of edges of the shortest path between vertices u and v in G . If no such path exists, $d_G(u, v) = \infty$.

Definition 1.3.25 (Diameter). The **diameter** of a graph G is

$$\text{diam}(G) = \max_{u, v \in V_G} d_G(u, v).$$

For a disconnected graph, the diameter is ∞ .

If a graph has finite diameter it is connected.

Definition 1.3.26 (Connected Graph). A graph G is **connected** if there exists a path between every pair of vertices in G . Otherwise, G is **disconnected**.

A graph can be further categorized according to whether multiple edges between a vertex and itself are permitted, whether there are infinite number of vertices and edges, whether the edges represent an unordered or ordered relationship between their endpoints, and whether the vertices are labelled or unlabelled.

Definition 1.3.27 (Simple Graph). A **simple graph** $G = (V, E)$ is a graph in which no two distinct edges connect the same pair of vertices and no edge connects a vertex to itself.

Definition 1.3.28 (Finite Graph). A **finite graph** is a graph $G = (V, E)$ for which both V and E are finite sets. A graph that is not finite is called an **infinite graph**.

Definition 1.3.29 (Directed Graph). A **directed graph** $G = (V, E)$ is a graph in which each edge $e \in E$ is an ordered pair of distinct vertices in V . Thus, (u, v) represents an edge directed from u to v , where (u, v) may not necessarily be equal to (v, u) .

An **undirected graph** $G = (V, E)$ is a graph in which each edge $e \in E$ is an unordered pair of distinct vertices in V . Thus, $\{u, v\} = \{v, u\}$ for every edge between vertices u and v .

Definition 1.3.30 (Labelled Graph). A **labelled graph** is a graph $G = (V, E)$ together with a labelling function $\ell : V \rightarrow L$, where L is a set of labels. A graph is **unlabelled** when no such vertex labels are specified.

In this thesis, we exclusively study finite, simple, undirected, unlabelled graphs. However, the constructions and algorithms may use additional structure, including labels or multiple edges, which must be analysed and restricted to their corresponding simple, unlabelled counterparts.

In particular, two graphs may have different vertex sets while nevertheless having the same underlying structure. This motivates the following notion of isomorphism.

Definition 1.3.31 (Graph Isomorphism). Let G and H be graphs. An **isomorphism** from G to H is a bijection

$$f : V_G \rightarrow V_H \quad \text{such that} \quad uv \in E_G \text{ if and only if } f(u)f(v) \in E_H.$$

We say that G is **isomorphic** to H , written $G \cong H$, if there exists an isomorphism from G to H .

The isomorphism relation is an equivalence relation on the set of graphs, and an **isomorphism class** is an equivalence class of graphs under this relation. Thus, graphs within the same isomorphism class have the same structure up to a relabelling of their vertices.

We will also consider graphs obtained by restricting the vertices and edges of another graph.

Definition 1.3.32 (Subgraph). A graph $H = (V_H, E_H)$ is a **subgraph** of a graph $G = (V_G, E_G)$ if

$$V_H \subseteq V_G \quad \text{and} \quad E_H \subseteq E_G.$$

For $S \subseteq V_G$, the **induced subgraph** of G on S , denoted by $G[S]$, is the graph with vertex set S and edge set

$$E_{G[S]} = \{uv \in E_G : u, v \in S\}.$$

One of the most important types of graphs is the complete graph, and its subgraph counterpart.

Definition 1.3.33 (Complete Graph). A graph on n vertices is **complete** if every pair of distinct vertices is adjacent. The complete graph on n vertices is denoted by K_n .

Definition 1.3.34 (Clique). A **clique** of a graph G is a subset $S \subseteq V_G$ such that the induced subgraph $G[S]$ is complete. The **clique number** of G , denoted by $\omega(G)$, is the size of a largest clique in G . A clique containing k vertices is denoted by K_k .

Definition 1.3.35. A **multicliqual vertex** is a vertex that belongs to multiple maximal cliques, an important distinction from just cliques since any vertex belonging to a clique of size n naturally belongs to a clique of size $n - 1$. A **unicliqual vertex** is a vertex that belongs to only one maximal clique.

In addition to being specified using a set of vertices and a set of edges, a graph can also be represented using a matrix.

Definition 1.3.36 (Adjacency Matrix). The **adjacency matrix** of graph H is denoted by $A_H \in \{0, 1\}^{n \times n}$ where

$$(A_H)_{ij} = \begin{cases} 1, & \text{if } ij \in E_H, \\ 0, & \text{otherwise,} \end{cases}$$

The degrees of the vertices can similarly be represented by a diagonal matrix.

Definition 1.3.37 (Degree Matrix). The **degree matrix** of graph H is denoted by $D_H \in \mathbb{R}^{n \times n}$ where

$$(D_H)_{ij} = \begin{cases} \deg_H(i), & \text{if } i = j, \\ 0, & \text{otherwise.} \end{cases}$$

In our discussion of graphs represented as matrices, we will find the following notation useful.

Definition 1.3.38 (Indicator Function). Let $\mathbb{1}_{\geq 1} : \mathbb{R}^{n \times n} \rightarrow \{0, 1\}^{n \times n}$ denote the entrywise **indicator function**, defined by

$$\mathbb{1}_{\geq 1}(M)_{ij} = \begin{cases} 1, & \text{if } M_{ij} \geq 1, \\ 0, & \text{otherwise.} \end{cases}$$

We now introduce the graph-theoretic objects central to this thesis.

Definition 1.3.39 (Graph Root). Let the **graph root** $H = (V, E_H)$ be a finite, simple, undirected graph, where V is the vertex set with $|V| = n$, and $E_H \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}$ is the edge set.

Definition 1.3.40 (Square of a Graph). The **square of a graph** H , denoted by $H^2 = (V, E_{H^2})$ is the graph on the same vertex set with edge set

$$E_{H^2} = \{ uv \mid u \neq v, 1 \leq d_H(u, v) \leq 2 \}. \quad (1.1)$$

That is, two distinct vertices are adjacent in H^2 if and only if they are connected by a path of length one or two in H . This is the standard definition of the square of a graph.

Definition 1.3.41 (Graph Square). The **graph square** of H is the undirected graph defined on the same vertex set $G = (V, E_G)$. Define the adjacency matrix of G using the adjacency and degree matrices of H

$$A_G = \mathbb{1}_{\geq 1}((A_H)^2 - D_H + A_H). \quad (1.2)$$

1.3.3 Mathematical Optimization

Mathematical optimization studies the problem of finding the best solution from all feasible solutions which are defined by a collection of constraints. An optimization problem usually consists of decision variables, an objective function to be maximized or minimized, and a set of constraints restricting the feasible values of the variables.

Definition 1.3.42 (Integer Program). An **integer program** (LP) is an optimization problem of the form

$$\begin{aligned} \max/\min \quad & c^\top x \\ \text{subject to} \quad & Ax \leq b, \\ & x \in \mathbb{Z}^n, \end{aligned}$$

where $x \in \mathbb{Z}^n$ is the vector of decision variables, $c \in \mathbb{R}^n$ is the objective coefficient vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, and $b \in \mathbb{R}^m$ is the constraint vector.

A linear relaxation is obtained by removing the integrality requirements from an integer program while retaining its linear objective function and constraints.

Definition 1.3.43 (Linear Relaxation). Let IP be an integer program. The **linear relaxation** of IP is the linear program obtained by removing the requirement that its variables take integer values.

Definition 1.3.44 (Linear Program). A **linear program** (LP) is an optimization problem of the form

$$\begin{aligned} \max/\min \quad & c^\top x \\ \text{subject to} \quad & Ax \leq b, \\ & x \in \mathbb{R}^n, \end{aligned}$$

where $x \in \mathbb{R}^n$ is the vector of decision variables, $c \in \mathbb{R}^n$ is the objective coefficient vector, $A \in \mathbb{R}^{m \times n}$ is the constraint matrix, and $b \in \mathbb{R}^m$ is the constraint vector.

Definition 1.3.45 (Feasible Solution). A **feasible solution** to an optimization problem is an assignment of values to its decision variables that satisfies all of its constraints. The set of all feasible solutions is called the **feasible set** and is denoted by $\mathcal{F}$. Geometrically, $\mathcal{F}$ is the subset of the decision-variable space satisfying all of the constraints and is referred to as the **feasible region**.

Among the feasible solutions, an optimal solution is one that achieves the best value of the objective function.

Definition 1.3.46 (Optimal Solution). An **optimal solution** is a feasible solution that maximizes or minimizes the objective function, depending on the optimization problem. The corresponding value of the objective function is called the **optimal value** and is denoted by OPT .

When an integer program is relaxed to a linear program, every feasible integer solution remains feasible for the relaxation. Consequently, if $\mathcal{F}_{IP}$ and $\mathcal{F}_{LP}$ denote the feasible sets of the integer program and its linear relaxation respectively, then

$$\mathcal{F}_{IP} \subseteq \mathcal{F}_{LP}.$$

Geometrically, $\mathcal{F}_{IP}$ is represented by the feasible integer points in the decision-variable space, whereas $\mathcal{F}_{LP}$ is the polyhedron defined by the relaxed constraints and may contain additional points.

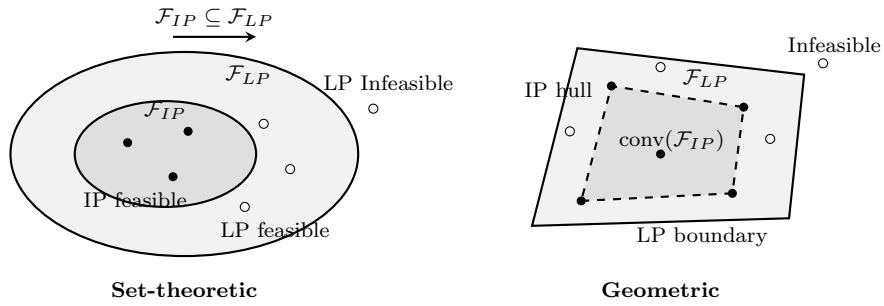


Figure 1.2: Set-theoretic and geometric relationships between an integer program and its linear relaxation.

For a maximization problem, this set inclusion implies $\text{OPT}_{IP} \leq \text{OPT}_{LP}$, so the linear relaxation provides an upper bound on the optimal value of the integer program. For a minimization problem, $\text{OPT}_{LP} \leq \text{OPT}_{IP}$, so the linear relaxation provides a lower bound.

Definition 1.3.47 (Integer Hull). Let $\mathcal{F}_{IP}$ denote the feasible set of an integer program. The **integer hull** is the convex hull of the feasible integer solutions, denoted by $\text{conv}(\mathcal{F}_{IP})$. That is, the integer hull is the set of all convex combinations of points in $\mathcal{F}_{IP}$.

Since the feasible region of a linear program is convex, the integer hull is contained in the feasible region of its linear relaxation. Hence,

$$\mathcal{F}_{IP} \subseteq \text{conv}(\mathcal{F}_{IP}) \subseteq \mathcal{F}_{LP}.$$

The integer hull is the smallest convex set containing all feasible integer solutions. Thus, geometrically, the gap between the integer hull and the LP feasible region represents the additional solutions admitted by the relaxation. If

$$\text{conv}(\mathcal{F}_{IP}) = \mathcal{F}_{LP},$$

then the linear relaxation coincides with the integer hull and is therefore an exact convex description of the feasible integer solutions.

Chapter 2

Algorithmic Approaches

In this chapter we explore different methodologies that have been used to show certain properties of the graph root and the graph square. For many restricted cases which are **NP**, a polynomial time reduction is given and in cases which are polynomial, a polynomial time algorithm is given.

2.1 NP-Completeness Proof

We now establish the computational complexity of the graph square problem as was done by Motwani and Sudan in their paper [MS94]. The decision version of the problem asks whether a given graph admits a graph root whose square is the given graph.

Problem 2.1.1 (SQUARE).

Instance: A finite, simple, undirected graph $G = (V, E)$.

Question: Does there exist a finite, simple, undirected graph H such that

$$G = H^2?$$

Theorem 1 (NP). *Problem (2.1.1) is in NP.*

NP Proof. Given a candidate square root H on n vertices, it is sufficient to verify that $G = H^2$. For each pair of distinct vertices $u, v \in V_H$, check whether u and v are adjacent in H or have a common neighbour in H . This determines whether $d_H(u, v) \leq 2$ and therefore whether $uv \in E_{H^2}$.

There are $\binom{n}{2} = \frac{n(n-1)}{2}$ unordered pairs of vertices. For each pair, determining whether the vertices have a common neighbour requires at most $O(n)$ operations. Hence, the total verification time is

$$O(n^2) \cdot O(n) = O(n^3).$$

By the definition of Big- O notation in Definition 1.3.1, this is polynomial in n . Therefore, the certificate H can be verified in polynomial time, and consequently

$$\text{SQUARE} \in \text{NP}.$$

□

Problem 2.1.2 (NAESAT).

Instance: A 3-CNF formula

$$\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

where each clause C_i is over variables $x_1, \dots, x_n$.

Question: Does there exist an assignment of TRUE and FALSE to the variables such that every clause contains at least one TRUE literal and at least one FALSE literal?

Thus, an assignment satisfies the not-all-equal condition if, for every clause, its literals are not all assigned the same truth value. The problem NAESAT is known to be **NP**-complete by Schaefer's dichotomy theorem [Sch78].

2.1.1 NP-Hard proof

Following [MS94], we show SQUARE is **NP**-hard by giving a polynomial-time reduction from **NP**-complete NAESAT and then show that G_ϕ has a graph root if and only if $\phi \in \text{NAESAT}$.

Polynomial Reduction. Let $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_m$ be an instance of NAESAT, where ϕ is a 3-CNF formula on variables $x_1, \dots, x_n$. Over variables $x_1, \dots, x_n$, construct a graph $G_\phi = (V_\phi, E_\phi)$ in polynomial time such that

$$\phi \in \text{NAESAT} \quad \text{if and only if} \quad G_\phi \text{ has a square root.}$$

The vertex set V_ϕ consists of

- (i) Two constant vertices TRUE and FALSE, plus a vertex x and three tail vertices t_1, t_2, t_3 for x .
- (ii) For each of the $2n$ literals $\ell_1, \dots, \ell_{2n}$, where $\ell_{i+n} = \overline{\ell_i}$, a literal vertex L_i .
- (iii) For every pair of literals ℓ_i, ℓ_k that are not complements of each other, a literal-pair vertex A_{ik} .
- (iv) For each clause $C_j = \{\ell_{j_1}, \ell_{j_2}, \ell_{j_3}\}$, a clause vertex C_j together with three tail vertices C_j^1, C_j^2, C_j^3 .

The edge set E_ϕ defined as

- i) **Tail of x .** The tail of x consists of the edges

$$t_3t_2, \quad t_3t_1, \quad t_2t_1, \quad t_2x, \quad t_1x, \quad t_1\text{TRUE}, \quad t_1\text{FALSE}.$$

- ii) **Tails of the clause vertices.** For every clause C_j , introduce the edges

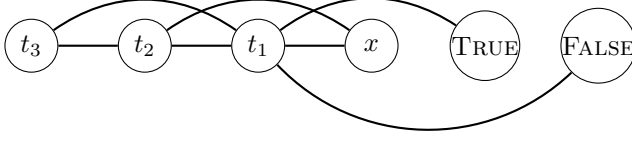
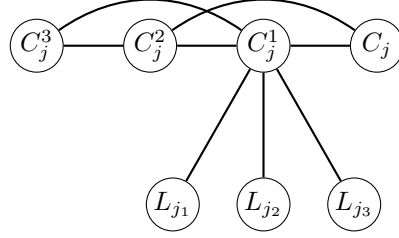
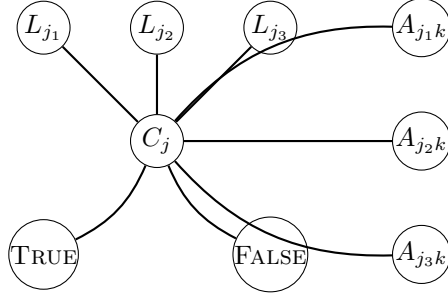
$$C_j^3C_j^2, \quad C_j^3C_j^1, \quad C_j^2C_j^1, \quad C_j^2C_j, \quad C_j^1C_j, \quad C_j^1L_{j_1}, \quad C_j^1L_{j_2}, \quad C_j^1L_{j_3},$$

where $C_j = \{\ell_{j_1}, \ell_{j_2}, \ell_{j_3}\}$.

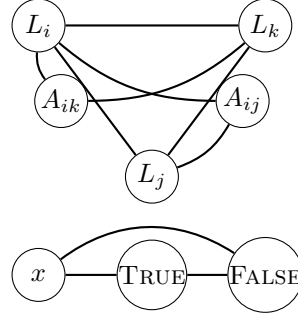
- iii) **Edges incident to clause vertices.** For every clause C_j and every literal $\ell_i \in C_j$, introduce the edge C_jL_i . Also introduce $C_j\text{TRUE}$ and $C_j\text{FALSE}$. Finally, for every literal ℓ_k that is not the complement of a literal in C_j , introduce the edge C_jA_{ik} .
- iv) **Edges among the remaining vertices.** All vertices in

$$\{L_i : 1 \leq i \leq 2n\} \cup \{A_{ik} : \ell_i \neq \overline{\ell_k}\} \cup \{\text{TRUE}, \text{FALSE}, x\}$$

are adjacent to one another, except for pairs of complementary literals. Thus, $L_iL_k \in E_\phi$ whenever ℓ_i and ℓ_k are not complementary, while $L_iL_k \notin E_\phi$ whenever ℓ_i and ℓ_k are complementary.

(i) Tail of x (ii) Tail of C_j (iii) Edges incident to C_j 

(iv) Remaining vertices



$\{L_i\} \cup \{A_{ik}\} \cup \{\text{TRUE}, \text{FALSE}, x\}$
is complete, except for complementary literal pairs

□

The construction is polynomial in $n + m$ and depends only on the formula ϕ . Therefore it can be carried out without knowing whether ϕ has a not-all-equal satisfying assignment. Moreover, the number of vertices and edges introduced is polynomial in the size of ϕ .

The following lemma is the central combinatorial tool for proving the correctness of the reduction.

Lemma 1 (Tail Lemma). *Let G be a connected graph containing four distinct vertices a, b, c, d such that*

$$N_G(a) = \{b, c\}, \quad N_G(b) = \{a, c, d\}, \quad cd \in E_G,$$

and suppose c has a neighbour in G outside $\{a, b, c, d\}$. If $H^2 = G$, then

$$N_H(d) \setminus \{a, b, c, d\} = N_G(c) \setminus \{a, b, c, d\}.$$

Proof. Since $N_G(a) = \{b, c\}$, every neighbour of a in H is contained in $\{b, c\}$.

First show that $N_H(a) = \{b\}$. Suppose, for contradiction, that $N_H(a) = \{c\}$. Then $ac \in E_H$. Because $ab \in E_G = E_{H^2}$, there is a path of length at most two from a to b in H . As a only has the neighbour c , this forces $bc \in E_H$.

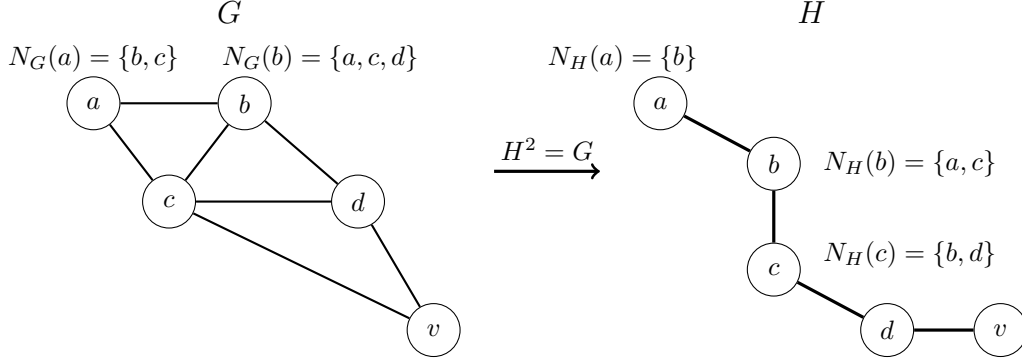


Figure 2.1: The forced local structure in the Tail Lemma. The square root H contains the path $a - b - c - d$. Consequently, $ac, bd \in E_G$ as distance-two edges. For an external neighbour v of c , the Tail Lemma gives $dv \in E_H$, and hence $dv \in E_G$.

Now $cd \in E_G$, so c and d are at distance at most two in H . As $bc \in E_H$, the only possible P_2 from c to d uses b , and hence $bd \in E_H$.

Let $v \notin \{a, b, c, d\}$ be the neighbour of c in G . Since $cv \in E_G$, there is a path of length at most two from c to v in H . This path cannot go through a , otherwise $av \in E_G$, contradicting $N_G(a) = \{b, c\}$. Nor can it go through b , since that would give $bv \in E_G$, contradicting $N_G(b) = \{a, c, d\}$. Therefore the path must be $c - d - v$, giving $dv \in E_H$.

Since $bd, dv \in E_H$, have a path $b - d - v$ of length two in H , which implies $bv \in E_G$. This again contradicts $N_G(b) = \{a, c, d\}$. The contradiction eliminates the possibility $N_H(a) = \{c\}$, and consequently

$$N_H(a) = \{b\}. \quad (2.1)$$

Now we determine the neighbourhood of c . Since $ab \in E_H$ as $N_H(a) = \{b\}$, and $ac \in E_G$, the edge ac in $G = H^2$ must be realized by the path $a - b - c$. This gives that $bc \in E_H$. If b had a neighbour $x \notin \{a, c\}$ in H , then $a - b - x$ would be a P_2 , forcing $ax \in E_G$, which contradicts $N_H(a) = \{b\}$. Thus

$$N_H(b) = \{a, c\}. \quad (2.2)$$

Since $bd \in E_G$ and $N_H(b) = \{a, c\}$, the P_2 from b to d in H must be $b - c - d$ - it cannot use a , as $N_H(a) = \{b\}$. Therefore $cd \in E_H$.

If c had a neighbour $y \notin \{b, d\}$ in H , then $b - c - y$ would give $by \in E_G$, contradicting Equation (2.2). Hence

$$N_H(c) = \{b, d\}. \quad (2.3)$$

It remains to relate the external neighbours of c in G to those of d in H . Take any $v \notin \{a, b, c, d\}$. If $cv \in E_G$, then there is a path of length at most two from c to v in H . By Equation (2.3), such a path must use b or d as the intermediate vertex. It cannot use b , since that would imply $bv \in E_G$, contradicting $N_G(b) = \{a, c, d\}$. Hence the path is $c - d - v$, so $dv \in E_H$. Conversely, if $dv \in E_H$, then $c - d - v$ is a P_2 in H since $cd \in E_H$, giving $cv \in E_G$.

Therefore have established that for every $v \notin \{a, b, c, d\}$,

$$v \in N_G(c) \text{ if and only if } v \in N_H(d).$$

The desired set equality follows immediately. $\square$

We now establish the two directions required for the validity of the reduction.

Lemma 2. *If ϕ is a **Yes-instance** of NAESAT, then G_ϕ has a graph root.*

Proof Sketch. Suppose that ϕ has an assignment satisfying not-all-equal. We construct a graph root $H = (V_\phi, E_H)$ on the same vertex set as G_ϕ as follows.

The edges of H consist of the edges of the tails, together with edges connecting x to TRUE and FALSE. For each clause vertex C_j , connect C_j to the three literal vertices corresponding to the literals contained in the clause, together with the edges required by its tail. Each literal vertex is connected to TRUE if its corresponding literal is assigned TRUE and to FALSE if its corresponding literal is assigned FALSE. Finally, each literal vertex is connected to the appropriate literal-pair vertices, and each literal-pair vertex is connected to both TRUE and FALSE.

Because the assignment is not-all-equal, every clause contains at least one TRUE literal and at least one FALSE literal. Consequently, the paths in H of length two produce precisely the edges of G_ϕ . In particular, complementary literals do not have a common neighbour in H , whereas every pair of non-complementary literals has a common neighbour through its corresponding literal-pair vertex. The tail construction similarly ensures that the required tail edges are obtained in H^2 .

Hence, $G_\phi = H^2$, and therefore G_ϕ has a graph root. $\square$

Lemma 3. *If G_ϕ has a graph root, then ϕ is a **Yes-instance** of NAESAT*

Proof Sketch. Suppose that G_ϕ has a graph root H , so that $G_\phi = H^2$.

Apply Lemma 1 to the tail of x , with $(a, b, c, d) = (t_3, t_2, t_1, x)$. Since t_1 has the external neighbours TRUE and FALSE in G_ϕ , the lemma gives

$$N_H(x) \setminus \{t_3, t_2, t_1, x\} = \{\text{TRUE}, \text{FALSE}\}.$$

Thus, apart from its own tail, x is adjacent in H precisely to TRUE and FALSE.

Similarly, for each clause C_j , apply Lemma 1 to its tail with $(a, b, c, d) = (C_j^3, C_j^2, C_j^1, C_j)$. Since C_j^1 has the external neighbours L_{j1}, L_{j2}, L_{j3} in G_ϕ , the lemma gives

$$N_H(C_j) \setminus \{C_j^3, C_j^2, C_j^1, C_j\} = \{L_{j1}, L_{j2}, L_{j3}\}.$$

Hence, apart from its own tail, C_j is adjacent in H precisely to the three literal vertices occurring in C_j . Since every literal vertex L_i is adjacent to x in $G_\phi = H^2$, each L_i must be adjacent in H to at least one of TRUE and FALSE.

On the other hand, complementary literal vertices are non-adjacent in G_ϕ . Therefore they cannot have a common neighbour in H , and hence a literal and its complement cannot both be adjacent to the same one of TRUE and FALSE. It follows that every literal vertex is adjacent to exactly one of TRUE and FALSE, and complementary literals are adjacent to opposite constant vertices.

We may therefore define an assignment by declaring a literal ℓ_i to be TRUE only when $L_i \text{TRUE} \in E_H$. Consider any clause $C_j = \{\ell_{j1}, \ell_{j2}, \ell_{j3}\}$. By construction, C_j is adjacent in G_ϕ to both TRUE and FALSE. Since, apart from its own tail, C_j is adjacent in H only to L_{j1}, L_{j2}, L_{j3} , these three literal vertices must include one adjacent to TRUE and one adjacent to FALSE.

Thus every clause contains at least one TRUE literal and at least one FALSE literal. Therefore the assignment is a not-all-equal satisfying assignment of ϕ , and hence

$$\phi \in \text{NAESAT}.$$

$\square$

Theorem 2 (NP-complete). *Problem (2.1.1) is NP-complete.*

Proof. Theorem (1) shows that Problem (2.1.1) belongs to **NP**. For **NP**-hardness, consider the polynomial reduction from NAESAT to SQUARE given in Proof (2.1.1). By Lemma (2), every **Yes**-instance ϕ of NAESAT produces a graph G_ϕ with a graph root. Together with Lemma (3), this gives that $\text{NAESAT} \leq_p \text{SQUARE}$, so Problem (2.1.1) is **NP**-hard.

Since Problem (2.1.1) belongs to **NP** and is **NP**-hard, it is **NP**-complete. $\square$

2.2 Algorithm for Tree Roots

Ross and Harary give a necessary and sufficient characterization of graphs that are squares of trees, together with an algorithm for constructing the corresponding tree square root [RH60]. The characterization is based on the maximal cliques of the square and the way in which these cliques intersect. We first establish the structural properties of the square of a tree that underlie the characterization, before describing the root finding algorithm.

Theorem 3 (Structural Characterization of Squares of Trees). *Let G be a finite, simple, undirected graph. Then $G = T^2$ for some tree T if and only if either G is a complete graph, or the maximal cliques $C_1, \dots, C_n$ of G satisfy the following conditions:*

- (i) *G is connected and the closed neighbourhood of every vertex of T forms a clique in G .*
- (ii) *If two maximal cliques have exactly one common vertex v , then there is another maximal clique containing v and one additional vertex from each of the two cliques.*
- (iii) *Each maximal clique C_i can be associated with a unique multicliqual vertex b_i . The number of multicliqual vertices contained in C_i is equal to the number of maximal cliques containing b_i .*
- (iv) *For any two maximal cliques C_i and C_j , $|C_i \cap C_j| \leq 2$, that is their intersection contains at most two points*
- (v) *The number of pairs of maximal cliques whose intersection contains two vertices is $n - 1$.*

To establish this characterization, let G be the square of a tree T and first consider several properties.

Lemma 4. *G is complete if and only if T is a star*

Proof. Suppose that T has a vertex v that is adjacent to every other vertex. Then every pair of vertices in T is at distance at most two, since any two vertices are either adjacent or have v as a common neighbour. Hence $G = T^2$ is complete.

Conversely, suppose that $G = T^2$ is complete. Then every pair of vertices in T has distance at most two, so $\text{diam}(T) \leq 2$. Since T is a tree, this implies that there is a vertex v adjacent to every other vertex. Thus, choosing v as the root gives a rooted tree T of depth one. $\square$

Lemma 5. *G is connected and the closed neighbourhood of every vertex of T forms a clique in G .*

Proof. Let $v \in V_T$. Every pair of vertices in the closed neighbourhood $N_T[v]$ are either adjacent in T or have v as a common neighbour and thus has distance at most two in T . Hence $N_T[v]$ forms a clique in $G = T^2$. Furthermore, every edge of T is also an edge of G , so T is a spanning subgraph of G . Since T is connected, this implies G is also connected. $\square$

Lemma 6. *If G is not complete, then the maximal cliques of G are precisely the closed neighbourhoods of the non-leaf vertices of T .*

Proof. Let v be a non-leaf vertex of T . Since v has at least two neighbours, every pair of vertices in $N_T(v)$ is at distance two through v . Hence $C(v) = N_T[v]$ is a clique in G . Now consider a vertex $u \notin N_T[v]$. The unique path from u to v in T has length at least two. If this path has length greater than two, then u is not adjacent in G to v . If the path has length two, then u can only be adjacent in G to the vertices along the corresponding part of the path. In either case, u cannot be added to $N_T[v]$ while maintaining a clique. Therefore $N_T[v]$ is a maximal clique.

Conversely, let C be a maximal clique of G . Since $G = T^2$, vertices in C are pairwise at distance at most two in T . The vertices of such a clique must therefore be contained in the closed neighbourhood of a common non-leaf vertex v of T . Thus $C \subseteq N_T[v]$. Since $N_T[v]$ is itself a clique and C is maximal, this gives $C = N_T[v]$. Hence every maximal clique of G is the closed neighbourhood of a non-leaf vertex of T . $\square$

Lemma 7. *If G is not complete, then a vertex $v \in V_G$ belongs to exactly one maximal clique if and only if v is a leaf of T .*

Proof. Let v be a leaf of T , and let u be its unique neighbour. By Lemma 6, every maximal clique corresponds to the closed neighbourhood of a non-leaf vertex. Since v has only one neighbour, the only maximal clique containing v is $N_T[u]$. Thus v belongs to exactly one maximal clique.

Conversely, suppose that v is not a leaf. Then v has at least two neighbours in T . By Lemma 6, $N_T[v]$ is a maximal clique containing v . Since G is not complete, Lemma 4 implies that T is not a star. Hence at least one neighbour u of v is a non-leaf vertex. The closed neighbourhood $N_T[u]$ is therefore another maximal clique containing v . Thus v belongs to at least two maximal cliques. Therefore, the vertices belonging to exactly one maximal clique are exactly the leaves of T . $\square$

Lemma 8. *Let C_i and C_j be maximal cliques corresponding to non-leaf vertices v_i and v_j of T . Then $|C_i \cap C_j| = 2$ if and only if $v_i v_j \in E_T$.*

Proof. Suppose that v_i and v_j are adjacent in T . Then both v_i and v_j belong to $N_T[v_i] \cap N_T[v_j]$. If there were another vertex u in this intersection, then u would be adjacent to both v_i and v_j in T . Together with the edge $v_i v_j$, this would create a cycle in T , which is impossible. Therefore, $|N_T[v_i] \cap N_T[v_j]| = 2$. By Lemma 6, these closed neighbourhoods are the corresponding maximal cliques, so $|C_i \cap C_j| = 2$.

Conversely, suppose that $|C_i \cap C_j| = 2$. Since v_i and v_j belong to their respective closed neighbourhoods, the intersection implies that v_i and v_j are either adjacent or have a common neighbour in T . If they were not adjacent, there would be a vertex u adjacent to both v_i and v_j . Then $v_i, v_j, u \in C_i \cap C_j$, which would give an intersection of at least three vertices, contradicting $|C_i \cap C_j| = 2$. Therefore, $v_i v_j \in E_T$. $\square$

Lemma 9. *G is a 2-edge-connected graph where the removal of any edge does not result in a disconnected graph*

Proof. Let $uv \in E_G$. Consider the edge uv and the graph $G - uv$. Since $G = T^2$, either $uv \in E_T$, or there exists a vertex $w \in V_T$ such that $uw, vw \in E_T$. In the latter case, u and v are connected by the path $u - w - v$ in $G - uv$.

If $uv \in E_T$, then the vertices closest to u and v along the remaining components of $T - uv$ are adjacent in G . Thus, the two components of $T - uv$ remain connected through vertices that are joined by edges in $G - uv$.

Consequently, removing uv from G does not disconnect the graph. Hence G has no bridges and is 2-edge-connected. $\square$

Combining gives the following proof of Theorem (3)

Proof. Suppose first that $G = T^2$ for some tree T . If G is complete, then Lemma 4 shows that T is a star.

Now suppose that G is not complete. By Lemma 5, G is connected and every closed neighbourhood in T forms a clique in G , giving (i).

By Lemma 6, each maximal clique of G is the closed neighbourhood of a non-leaf vertex of T . Suppose that two maximal cliques $N_T[u]$ and $N_T[v]$ have exactly one common vertex x . Since T is a tree, u and v cannot be adjacent, and hence x lies on the unique path between u and v . Thus, x is adjacent in T to both u and v , so $N_T[x]$ forms a maximal clique containing x , together with the additional vertices u and v from the two original cliques. This gives (ii).

For a maximal clique $C_i = N_T[b_i]$, the associated vertex b_i is the corresponding non-leaf vertex of T . Every non-leaf neighbour of b_i in T gives another maximal clique containing b_i , while each such neighbour is itself also a multicliqual vertex contained in C_i . Consequently, the number of multicliqual vertices contained in C_i is equal to the number of maximal cliques containing b_i , giving (iii).

By Lemma 8, two maximal cliques intersect in two vertices exactly when their corresponding non-leaf vertices are adjacent in T . Since T contains no cycles, two such closed neighbourhoods cannot have more than two vertices in common. This gives (iv). Finally, the non-leaf vertices of T , together with the edges between them, form a tree. If there are n such vertices, there are exactly $n - 1$ edges between them. By Lemma 8, each such edge corresponds to a pair of maximal cliques with an intersection containing exactly two vertices. Hence (v) holds.

Conversely, suppose that G is not complete and that its maximal cliques $C_1, \dots, C_n$ satisfy (i)–(v). By (iii), we may associate each maximal clique C_i with a unique vertex b_i that belongs to multiple maximal cliques. These vertices will form the internal vertices of the tree root. Construct T with vertex set $V_T = \{b_1, \dots, b_n\}$. For every pair of maximal cliques satisfying $|C_i \cap C_j| = 2$, add the edge $b_i b_j$ to T . By (v), T contains exactly $n - 1$ edges, and the intersection structure given by the characterization ensures that these edges connect all vertices without forming a cycle. Thus T is a tree. It remains to add the unicliqual vertices. By (iii), these vertices correspond to leaves of the root. For each clique C_i , attach all vertices of C_i that occur in no other maximal clique as leaves adjacent to b_i . The resulting graph is a tree T . Each maximal clique C_i is represented by the closed neighbourhood of b_i in T , while the remaining vertices are attached as leaves to their corresponding internal vertices. Therefore every edge of G is represented by two vertices at distance at most two in T , and no additional edges are introduced. Hence $T^2 = G$ and G is the square of a tree. $\square$

The proof of Theorem (3) gives the following algorithm. Given a graph known to be the square of a tree, identify all maximal cliques, classify the vertices as multicliqual or unicliqual according to the number of maximal cliques in which they occur, and construct the tree T by connecting the multicliqual vertices to form the internal structure of T and attaching the unicliqual vertices as leaves to their corresponding multicliqual vertices.

Theorem 4 (Tree Root Algorithm). *Given a graph $G = T^2$, Algorithm (1) returns a tree T such that $T^2 = G$. Furthermore, this tree is unique up to isomorphism.*

Proof. If G is complete, Step 1 returns a star. By Lemma 4, this is a tree square root of G . Otherwise, Lemma 6 shows that each maximal clique corresponds to a non-leaf vertex of the root.

Algorithm 1 Ross-Harary Tree Construction

1: **Input:** A graph G where $G = T^2$

2: **Output:** A tree T

Step 1

3: **if** $G = K_p$ is complete **then**

4: **Return** tree T with depth 1 where root can be any vertex

5: **else**

6: **Continue**

7: **end if**

Step 2

8: Identify all maximal cliques $C_1, \dots, C_n$ of G using an algorithm such as Bron-Kerbosch [BK73] where $n > 1$ from characterization (i)

Step 3

9: Let $b_1, \dots, b_n$ be multicliqual points corresponding to $C_1, \dots, C_n$ formed from characterization (iii)

10: Construct a graph T with vertex set $\{b_1, \dots, b_n\}$

11: **for** each pair (C_i, C_j) satisfying $|C_i \cap C_j| = 2$ **do**

12: Add edge between b_i and b_j in T

13: **end for**

14: By characterization (v), T is a tree

Step 4

15: **for** each clique C_i **do**

16: Let n_i be the number of unicliqual points in C_i

17: Attach n_i leaf nodes to b_i in T

18: **end for**

19: The resulting tree T has the same number of nodes as G and satisfies $T^2 = G$

Thus the vertices $b_1, \dots, b_n$ identified in Step 3 determine the internal vertices of T . By Lemma 8, two of these internal vertices are adjacent precisely when their corresponding maximal cliques have an intersection of size two. Therefore Step 3 recovers all edges between the internal vertices. The remaining vertices belong to exactly one maximal clique and, by Lemma 7, correspond to leaves. Step 4 therefore places each of these vertices at its unique position in the tree. Hence the algorithm reconstructs a tree T satisfying $T^2 = G$. Since the internal vertices, their adjacencies, and the attachment of every leaf are all determined by the maximal cliques of G , the resulting tree is unique up to isomorphism. $\square$

While Algorithm 1 formalizes the original 1960 constructive proof by Ross and Harary, it pre-dates the formalization of computational complexity classes $\mathcal{P}$ vs. $\mathcal{NP}$ established in the early 1970s. From a modern perspective, Ross and Harary’s strategy relies on global maximal clique enumeration to group vertices into local star subgraphs. If implemented using general-purpose clique algorithms such as Bron–Kerbosch, this approach suffers from exponential worst-case execution time.

In contrast, Lin and Skiena bypass clique enumeration entirely by shifting to a local, bottom-up leaf-trimming strategy to give a polynomial algorithm [LS95]. Rather than searching for global subgraphs, they identify leaves of T directly as simplicial vertices with minimal degree, partition their closed neighborhoods into leaf and non-leaf sets, and use the structural properties of these local neighborhoods to reconstruct parent-child adjacencies directly. By repeatedly trimming identified leaves, they reduce the problem to smaller tree squares until the tree structure is fully recovered.

2.3 Mathematical Optimization Approach

The two methods so far have primarily focused on the definition of a square of a graph to be paths of length smaller than two defined in Definition (1.3.40). Using adjacency matrices, this definition is equivalent expressed as

$$A_G = (A_H)^2 \vee A_H. \quad (2.4)$$

where $\vee$ is applied entrywise to the matrices. This concise interpretation with the logical operator lends itself easily to a SAT problem whereby one could introduce Boolean variables for the edges of H and encode $(A_H)_{ij}^2 = ((A_H)_{i1} \wedge (A_H)_{1j}) \vee ((A_H)_{i2} \wedge (A_H)_{2j}) \vee \dots \vee ((A_H)_{in} \wedge (A_H)_{nj})$. Thus, for each pair of distinct vertices i, j , the condition that ij is an edge of G can be expressed as

$$g_{ij} \text{ if and only if } \left(\bigwedge_k (x_{ik} \wedge x_{kj}) \vee x_{ij} \right). \quad (2.5)$$

The resulting Boolean formula can be converted into a standard SAT instance using auxiliary variables and a polynomial number of clauses. Hence, the logical formulation provides a polynomial-size SAT representation of the graph-square condition.

However, constructing a polynomial-size SAT instance does not imply that the underlying problem can be solved in polynomial time. A general SAT instance may require exponential time to solve, since SAT is NP -complete. Therefore, to obtain a polynomial-time algorithm through this approach, one would additionally need to show that the SAT instances arising from Equation (2.5) belongs to a tractable subclass of SAT, such as 2-SAT, or otherwise provide a polynomial-time algorithm for solving these particular instances. The formulation in Equation (2.5) evidently does not belong to 2-SAT, as each constraint involves a disjunction over all possible common neighbours k and therefore contains terms involving more than two Boolean variables. More generally, one would suspect from the results in 2.1 that there is no immediate polynomial-time algorithm for

solving this instance and any restricted version of the graph root finding problem would necessitate significant machinery.

Instead, we aim to situate the problem completely within a polynomial-time framework by considering another equivalent representation of Equation (2.4) and its equivalent formulation in Definition (1.3.40).

Lemma 10. *The graph G defined by Definition (1.3.41) is equivalent to the graph square H^2 defined in Definition (1.3.40)*

$$ij \in E_{H^2} \text{ if and only if } 1 \leq d_H(i, j) \leq 2. \quad (2.6)$$

Proof. For $i \neq j$, since D_H is diagonal, have $(D_H)_{ij} = 0$ and hence

$$((A_H)^2 - D_H + A_H)_{ij} = (A_H)_{ij}^2 + (A_H)_{ij}.$$

Suppose $ij \in E_G$ such that $(A_H)_{ij}^2 + (A_H)_{ij} \geq 1$. This implies that at least one of $(A_H)_{ij} = 1$ or $(A_H)_{ij}^2 \geq 1$ must be true. If $(A_H)_{ij} = 1$, then $d_H(i, j) = 1$. If $(A_H)_{ij}^2 \geq 1$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $d_H(i, j) \leq 2$. Therefore $1 \leq d_H(i, j) \leq 2$, and thus $ij \in E_{H^2}$.

Suppose $ij \in E_{H^2}$ such that $1 \leq d_H(i, j) \leq 2$. D_H is diagonal such that $(D_H)_{ij} = 0$. If $d_H(i, j) = 1$, then $(A_H)_{ij} = 1$. If $d_H(i, j) = 2$, then there exists a vertex k such that $ik, kj \in E_H$, and hence $(A_H)_{ij}^2 \geq 1$. In either case, $(A_H)_{ij}^2 + (A_H)_{ij} \geq 1$, so $(A_G)_{ij} = 1$, and hence $ij \in E_G$.

For $i = j$, since all graphs are simple, have $(E)_{ii} = 0$, $(A)_{ii} = 0$ and hence

$$((A_H)^2 - D_H + A_H)_{ii} = (A_H)_{ii}^2 - (D_H)_{ii}.$$

$(A_H)_{ii}^2 - (D_H)_{ii} = 0$ is always true since the degree matrix of H always has diagonal values $(D_H)_{ii} = \sum_k x_{ik}$ and each edge from vertex i to its neighbour in H will always create a loop

to itself in the square such that $(A_H)_{ii}^2 = \sum_{k=1} x_{ik} x_{ki} = \sum_{k=1} x_{ik}^2$ by the symmetry of adjacency

matrix A_H . Since $x_{ik} \in \{0, 1\}$, the square of the boolean is identical to the boolean x_{ik} such that $(A_H)_{ii}^2 = \sum_k x_{ik} = (D_H)_{ii}$ and $(A_G)_{ii} = \mathbb{1}_{\geq 1}((A_H)^2 - D_H + A_H)$ as required.

Combining both gives that $E_G = E_{H^2}$, and so $G = H^2$. $\square$

The formulation of the adjacency matrix of the graph square in Equation (1.2) is very powerful. However, the presence of the indicator function introduces a nonlinearity that prevents the problem from being expressed as a purely quadratic form, thereby limiting the direct applicability of existing results from the quadratic optimization literature.

Lemma 11. *If H has girth 5, then Equation (1.2) can be simplified to become*

$$A_G = ((A_H)^2 - D_H + A_H). \quad (2.7)$$

Proof. Since the graph root has girth 5 there are no cycles of length 3 or 4 in H . Thus for every pair of vertices $i \neq j \in V$ there is either no cycle that contains both i and j or i and j lay on a common cycle of length at least 5. In all cases $(D_H)_{ij} = 0$. If there is no cycle, then there is at most one path between i, j

- i) If the path is of length 1, then $(A_H)_{ij} = 1$, $(A_H)_{ij}^2 = 0$ and so $(A_G)_{ij} = 0 - 0 + 1 = 1$.
- ii) If the path is of length 2, then $(A_H)_{ij} = 0$, $(A_H)_{ij}^2 = 1$ and so $(A_G)_{ij} = 1 - 0 - 0 = 1$.

iii) If the path is of length ≥ 3 or a path does not exist, then $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0$.

If i and j lay on a common cycle of length at least five then there are two paths between i, j

i) If one path is of length 1, then the other is of length ≥ 4 and hence $(A_H)_{ij} = 1, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0 - 0 + 1 = 1$.

ii) If one path is of length 2, then the other is of length ≥ 3 and hence $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 1 \implies (A_G)_{ij} = 1 - 0 - 0 = 1$.

iii) If the paths are both of length ≥ 3 , then $(A_H)_{ij} = 0, (A_H)_{ij}^2 = 0 \implies (A_G)_{ij} = 0$.

If $i = j$, then $(A_H)_{ii} = 0, (D_H)_{ii} = \deg_H(i)$, and $(A_H)_{ii}^2 = \deg_H(i)$. Thus,

$$(A_H)_{ii}^2 - (D_H)_{ii} + (A_H)_{ii} = \deg_H(i) - \deg_H(i) + 0 = 0 = (A_G)_{ii}.$$

□

We will not focus on this particular restricted class and analyze the integer program generated by Equation (2.7), but rather aim for a unified method for solving the problem in general. Nevertheless this result gives strong intuition into the results established by Farzad, Lau, Le, and Tuy in [Far+09] in constructing a polynomial time algorithm for recognizing graph roots where H has girth at least 6.

2.3.1 Formulation

Using Lemma (10), we construct an integer program to solve instances of SQUARE. The set of decision variables is

$$X = \{x_{ij} \mid i, j \in V_G\},$$

where $x_{ij} \in \{0, 1\}$ denotes the corresponding entry of the adjacency matrix A_H . In a feasible solution, $x_{ij} = 1$ if and only if $ij \in E_H$. The variables $y_{ikj} \in \{0, 1\}$ represent the products $x_{ik}x_{kj}$ in $(A_H)_{ij}^2$, while $z_{ij} \in \mathbb{Z}_{\geq 0}$ represents the argument of the indicator function.

Given an input G to SQUARE, our goal is the construct the following matrices, where g_{ij} is the corresponding entry of the adjacency matrix of G .

$$D_H = \begin{bmatrix} \deg_H(1) & 0 & \cdots & 0 & 0 \\ 0 & \ddots & \vdots & \ddots & \vdots \\ \vdots & \cdots & \deg_H(i) & \cdots & 0 \\ 0 & \ddots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & \deg_H(n) \end{bmatrix}, A_H = \begin{bmatrix} x_{11} & \cdots & x_{1j} & \cdots & x_{1n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{i1} & \cdots & x_{ij} & \cdots & x_{in} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{nj} & \cdots & x_{nn} \end{bmatrix} \text{ such that}$$

$$(A_H)^2 = \begin{bmatrix} \sum_{k=1}^n x_{1k}x_{k1} & \cdots & \sum_{k=1}^n x_{1k}x_{kj} & \cdots & \sum_{k=1}^n x_{1k}x_{kn} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \sum_{k=1}^n x_{ik}x_{k1} & \cdots & \sum_{k=1}^n x_{ik}x_{kj} & \cdots & \sum_{k=1}^n x_{ik}x_{kn} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \sum_{k=1}^n x_{nk}x_{k1} & \cdots & \sum_{k=1}^n x_{nk}x_{kj} & \cdots & \sum_{k=1}^n x_{nk}x_{kn} \end{bmatrix}.$$

Then with known constants $g_{ij} \in \{0, 1\}$ from A_G , can reformulating (1.2) as an integer program by equating entries of each matrix to give the following linear system of n^2 equations.

$$\begin{bmatrix} g_{11} \\ \vdots \\ g_{1j} \\ \vdots \\ g_{1n} \\ \vdots \\ g_{ij} \\ \vdots \\ g_{nn} \end{bmatrix} = \begin{bmatrix} \mathbb{1}_{\geq 1}(z_{11}) \\ \vdots \\ \mathbb{1}_{\geq 1}(z_{1j}) \\ \vdots \\ \mathbb{1}_{\geq 1}(z_{1n}) \\ \vdots \\ \mathbb{1}_{\geq 1}(z_{ij}) \\ \vdots \\ \mathbb{1}_{\geq 1}(z_{nn}) \end{bmatrix} = \begin{bmatrix} \mathbb{1}_{\geq 1}\left(\sum_{k=1}^n x_{1k}x_{k1} - (D_H)_{11} + x_{11}\right) \\ \vdots \\ \mathbb{1}_{\geq 1}\left(\sum_{k=1}^n x_{1k}x_{kj} - (D_H)_{1j} + x_{1j}\right) \\ \vdots \\ \mathbb{1}_{\geq 1}\left(\sum_{k=1}^n x_{1k}x_{kn} - (D_H)_{1n} + x_{1n}\right) \\ \vdots \\ \mathbb{1}_{\geq 1}\left(\sum_{k=1}^n x_{ik}x_{kj} - (D_H)_{ij} + x_{ij}\right) \\ \vdots \\ \mathbb{1}_{\geq 1}\left(\sum_{k=1}^n x_{nk}x_{kn} - (D_H)_{nn} + x_{nn}\right) \end{bmatrix}. \quad (2.8)$$

The matrix D_H is diagonal can be represented as $(D_H)_{ii} = \sum_k x_{ik}$ where $(D_H)_{ij} = 0$ for $i \neq j$.

The known z_{ij} represent the terms inside the indicator function such that

$$g_{ij} = 1 \text{ if and only if } z_{ij} \geq 1 \text{ and } g_{ij} = 0 \text{ if and only if } z_{ij} = 0.$$

Let $M \in \mathbb{Z}_{>0}$ be a sufficiently large constant. In this case, M is bounded by the number of edges such that it can be at most $n(n-1) = O(n^2)$. The above condition can be modelled using a standard big- M formulation, and the resulting integer program is given by

Variables:

$$x_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.9)$$

$$y_{ikj} \in \mathbb{Z}_{\geq 0}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.10)$$

$$z_{ij} \in \mathbb{Z}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.11)$$

Constraints:

$$(1) \text{ Indicator argument: } z_{ii} = \sum_{k=1}^n y_{iki} - \sum_{k=1}^n x_{ik}, \forall i \in \{1, \dots, n\}, \quad (2.12)$$

$$z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij}, \quad \forall i \neq j \in \{1, \dots, n\}, \quad (2.13)$$

$$(2) \text{ Indicator function: } z_{ij} \leq M g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.14)$$

$$z_{ij} \geq g_{ij}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.15)$$

$$(3) \text{ Product: } y_{ikj} \leq x_{ik}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.16)$$

$$y_{ikj} \leq x_{kj}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.17)$$

$$y_{ikj} \geq x_{ik} + x_{kj} - 1, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.18)$$

$$(4) \text{ Upper Bound: } x_{ij} \leq 1, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.19)$$

$$y_{ikj} \leq 1, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (2.20)$$

$$(5) \text{ Symmetry: } x_{ij} = x_{ji}, \quad \forall i, j \in \{1, \dots, n\}, \quad (2.21)$$

$$(6) \text{ Diagonal: } x_{ii} = 0, \quad \forall i \in \{1, \dots, n\}. \quad (2.22)$$

The construction of the integer program on the arbitrary graph G using Equations (2.9)–(2.22) is denoted as $\mathcal{IP}(G)$.

The constraints in Equations (2.21)–(2.22) impose the required symmetry and diagonal structure on the adjacency matrix A_H . Although analogous constraints could also be imposed on the auxiliary variables z , they are omitted as they are redundant and may introduce artificial degeneracy into the linear program.

2.3.2 Validity

Theorem 5. *Let G be a graph and consider the integer program whose variables $x_{ij} \in \{0, 1\}$ encode a graph H with adjacency matrix $A_H = (x_{ij})$. Then G has a square root H if and only if the integer program has a feasible solution x such that the corresponding graph H satisfies $H^2 = G$.*

Proof. Suppose G has a square root H and let A_H be the adjacency matrix of H . $x_{ij} = (A_H)_{ij}$. Construct the variables x_{ij}, y_{ikj}, z_{ij} and the constant g_{ij} as such

i) Let $x_{ij} = 1$ if $ij \in E_H$ and $= 0$ otherwise.

ii) Let $y_{ikj} = x_{ik}x_{kj}$ for all i, k, j .

iii) Let $z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij}$ for all i, j .

- iv) Let $z_{ii} = \sum_{k=1}^n y_{iki} - \sum_{k=1}^n x_{ik} = \sum_{k=1}^n x_{ik}^2 - \sum_{k=1}^n x_{ik} = 0$ since $x_{ij} \in \{0, 1\} \implies x_{ik}^2 = x_{ik}$.
- v) Let $g_{ij} = 1$ if $ij \in E_G$ and $= 0$ otherwise.

Verifying constraints of the integer program for the variables

- i) As $x_{ij} \in \{0, 1\} \implies y_{ikj} \in \{0, 1\}$, Equations (2.9)-(2.11) and Equations (2.19)-(2.20) are satisfied.
- ii) As H is a simple directed graph, its adjacency matrix is hallow and symmetric such that Equations (2.21) -(2.22) are satisfied.
- iii) As $y_{ikj} = x_{ik}x_{kj}$ and all variables are binary, Equations (2.16)- (2.18) are satisfied.
- iv) Equation (2.13) by definition is satisfied.
- v) As $z_{ii} = \sum_{k=1}^n x_{ik}x_{ki} - \sum_{k=1}^n x_{ik}$, Equation (2.12) is satisfied.
- vi) As $z_{ij} \geq 1$ if and only if $x_{ij} = 1$ or there exists k such that $x_{ik} = x_{kj} = 1$ and as $H^2 = G$, have $g_{ij} = 1$ if and only if $1 \leq d_H(i, j) \leq 2$. Thus $g_{ij} = 1$ if and only if $z_{ij} \geq 1$ so the indicator constraints Equations (2.14)-(2.15) hold.

All constraints of the integer program are satisfied by the constructed variables. Hence, the integer program is feasible.

Suppose the integer program on graph G has a feasible solution x , and let x_{ij} define a graph H by

$$ij \in E_H \text{ if and only if } x_{ij} = 1.$$

As x_{ij} satisfies the constraints of the integer program,

- i) Equation (2.9) and Equation (2.19) gives $x_{ij} \in \{0, 1\}$ for all i, j ,
- ii) Equation (2.21) and Equation (2.22) give $x_{ij} = x_{ji}$ and $x_{ii} = 0$,

such that x_{ij} defines a hollow, symmetric matrix with all elements being either 0 or 1 which is a valid adjacency matrix of a graph. Examining the auxiliary variables y_{ikj} and z_{ij} ,

- i) Equations (2.16)-(2.18) give the strict equality $y_{ikj} = x_{ik}x_{kj}$ for all i, j, k ,
- ii) Equation (2.13) gives $z_{ij} = \sum_{k=1}^n y_{ikj} + x_{ij} = \sum_{k=1}^n x_{ik}x_{kj} + x_{ij}$ for all $i \neq j$,
- iii) Equation (2.12) gives $z_{ii} = \sum_{k=1}^n y_{iki} + \sum_{k=1}^n x_{ik} = \sum_{k=1}^n x_{ik}x_{ki} - \sum_{k=1}^n x_{ik} = 0$ for all i since

$$\sum_{k=1}^n x_{ik}x_{ki} = \sum_{k=1}^n x_{ik}^2 = \sum_{k=1}^n x_{ik},$$

by symmetry and since $x_{ij} \in \{0, 1\}$.

The indicator constraints Equations (2.14)-(2.15) thus give for $i \neq j$

$$g_{ij} = 1 \text{ if and only if } \sum_{k=1}^n x_{ik}x_{kj} + x_{ij} \geq 1 \text{ and } g_{ij} = 0 \text{ if and only if } \sum_{k=1}^n x_{ik}x_{kj} + x_{ij} = 0.$$

Since $x_{ij} \in \{0, 1\}$, this is equivalent to

$$g_{ij} = 1 \text{ if and only if } (x_{ij} = 1) \vee \left(\text{there exists } k \text{ such that } x_{ik} = 1 \wedge x_{kj} = 1 \right) \text{ and } g_{ij} = 0 \text{ if and only if } x_{ij} = 0.$$

By definition of H this means that $g_{ij} = 1$ if and only if $(ij \in E_H) \vee \left(\text{there exists } k \text{ such that } ik, kj \in E_H \right)$ which is equivalent to

$$g_{ij} = 1 \text{ if and only if } 1 \leq d_H(i, j) \leq 2.$$

Applying Lemma 10 thus gives $ij \in E_G$ if and only if $ij \in E_H^2$ such that $G = H^2$. □

n	H exists	H does not exist
3	3	1
4	6	5
5	11	23
6	28	128
7	77	967
8	307	12039

Table 2.1: Outcomes of the integer program for graphs of different orders of n .

The correctness of the integer program is elucidated by Table (2.1). The total number of simple graphs on $n = 2, 3, 4, \dots$ unlabeled nodes is the sequence 2, 4, 11, 34, 156, 1044, 12346, $\dots$ given by A000088. The number of graphs which are squares is the sequence 2, 3, 6, 11, 28, 77, 307, $\dots$ given by A382181.

Chapter 3

Square of a Tree using Linear Programming

3.1 Linear Program Relaxation

The linear relaxation of the integer program defined by Equations (2.9)–(2.22) is constructed in order to generate a feasible weighted solution in polynomial time. It is done with identical constraints, however the decision variables (2.9), (2.10) and (2.11) are replaced with

$$x_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (3.1)$$

$$y_{ikj} \in \mathbb{R}_{\geq 0}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (3.2)$$

$$z_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}. \quad (3.3)$$

The construction of the linear program on the arbitrary graph G using Equations (3.1)–(3.3) and Equations (2.12)–(2.22) is denoted as $\mathcal{LP}(G)$.

The results from the linear programming can give *integral solutions* and *fractional solutions* when the root exists and *fractional solutions* and *infeasible solutions* when the root does not exist such that the results in Figure (3.1) can be generated.

Definition 3.1.1 (Integral Solutions). *A feasible solution to the linear program is an integral solution if*

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in V.$$

In this case, the variables x_{ij} define a graph root H through

$$E_H = \{(i, j) \in V \times V : x_{ij} = 1\}.$$

Definition 3.1.2 (Fractional Solutions). *A feasible solution to the linear program is a fractional solution if*

$$x_{ij} \in [0, 1] \quad \forall i, j \in V,$$

and there exists at least one pair (i, j) such that $0 < x_{ij} < 1$. In this case, the variables x_{ij} do not directly define a graph root.

Figure (3.1) can be equivalently represented in the more compact Table (3.1)

⁰The LP integral solution sequence 2, 3, 6, 10, 17, 29, 51, ... seems to be given by A285665.

¹The LP integral solution sequence 2, 3, 6, 10, 17, 29, 51, ... seems to be given by A285665.

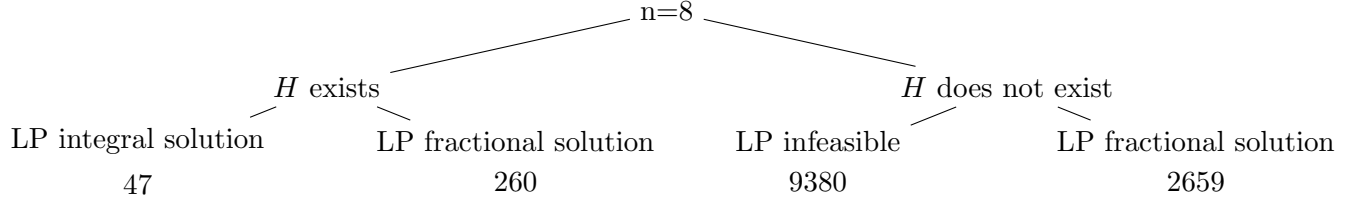


Figure 3.1: LP outcomes for different values for $n = 8$ on base linear program.

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	9	2	20	3
6	15	13	107	21
7	25	52	767	200
8	47	260	9380	2659

Table 3.1: Outcomes of the LP relaxation for graphs of different orders of n , classified according to whether a square root H exists.

1

In Figure 1.2 solutions which are LP feasible may be integral or fractional. The results in Table 3.1 demonstrate both cases. When H exists, an integral LP solution corresponds to a feasible point in $\mathcal{F} * IP$, while a fractional LP solution lies in $\mathcal{F} * LP \setminus \mathcal{F}_{IP}$. For $n = 3$ and $n = 4$, all instances for which H exists have integral LP solutions. From $n = 5$ onwards, fractional solutions begin to occur, with their number increasing as n increases.

When H does not exist, two outcomes are observed. An LP -infeasible instance has no point in $\mathcal{F} * LP$, and therefore cannot contain a feasible integer solution. Alternatively, the LP may remain feasible but return a fractional solution, placing the solution in $\mathcal{F} * LP \setminus \mathcal{F} * IP$. The number of such fractional solutions also increases with n .

These results demonstrate that LP feasibility alone is insufficient to establish the existence of a graph root, since $\mathcal{F}_{LP}$ may contain both integral and fractional solutions. Fractional solutions arise because the linear relaxation weakens the integrality restrictions of the integer program, allowing variables that are binary in the integer formulation to take values in the continuous interval $[0, 1]$.

Consider Equations (2.16)-(2.18) which gives $y_{ikj} = x_{ik}x_{kj}$. When x is binary, this construction of y leads to y being binary as well.

Consider Equation (2.12), where $z_{ii} = \sum_{k=1}^n y_{iki} - \sum_{k=1}^n x_{ik} + x_{ii}$ can be simplified to $z_{ii} = \sum_{k=1}^n x_{ik}^2 - \sum_{k=1}^n x_{ik}$ and is equal to 0 when x is binary. In the linear relaxation this strict inequality no longer holds for fractional values of x_{ik} . However, when x_{ik} is fractional $x_{ik}^2 \leq x_{ik}$ such that $z_{ii} = \sum_{k=1}^n x_{ik}^2 - x_{ik} \leq 0$. As $z \in \mathbb{Z}_{\geq 0}$, this gives that $z_{ii} = 0$ as required - no additional modifications are required.

Consider Equation (2.13), where $z_{ij} = \sum_{k=1}^n x_{ik}x_{kj} + x_{ij}$. When $z_{ij} = 0$, the equation follows

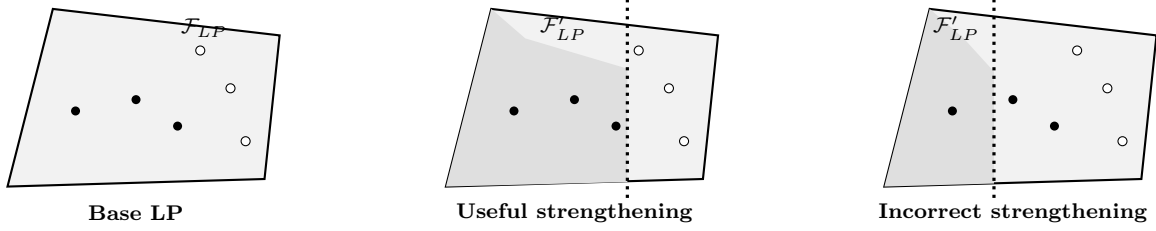


Figure 3.2: Strengthening the linear program with an additional constraint. The same feasible region and solutions are shown in each case, while the position of the additional constraint determines which solutions remain feasible. A useful constraint removes fractional solutions while retaining the integer solutions, whereas an overly restrictive constraint also removes valid integer solutions.

the standard definition of graph squares such that $x_{ij} = 0$ and for all intermediary ks at least one of x_{ik}, x_{kj} must be equal to 0. However, when $z_{ij} \geq 1$, fractional solutions of x_{ij} and $x_{ik}x_{kj}$ may satisfy the constraints without giving a valid integral path in the root. Thus, the linear relaxation may remain feasible even when no graph root exists.

The following proposition establishes the complementary case - whenever a graph root exists, the linear relaxation will never return an infeasible solution.

Proposition 1. *If G has a square root H , then $\mathcal{LP}(G)$ is feasible.*

Proof. Suppose that G has a square root H . By Theorem 5, the adjacency matrix A_H gives a feasible solution to $\mathcal{IP}(G)$. Since $\mathcal{LP}(G)$ is obtained by relaxing the integrality constraints of $\mathcal{IP}(G)$, every feasible solution of $\mathcal{IP}(G)$ is also feasible for $\mathcal{LP}(G)$. Hence $\mathcal{LP}(G)$ is feasible. $\square$

3.2 Min Linear Program

Consider the addition of an objective function maximizing the total weights on all edges to the linear relaxation defined in Section 3.1

$$\min \sum_{i=1}^n \sum_{j=1}^n x_{ij}. \quad (3.4)$$

The construction of the linear program on the arbitrary graph G using Equations (3.1)–(3.3), Equations (2.12)–(2.22) together with Equation (3.4) is denoted as $\min \mathcal{LP}(G)$.

This relaxation however yields more fractional solutions, as illustrated in Table 3.2.

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	2	1	1	0
4	3	3	5	0
5	3	8	20	3
6	4	24	107	21
7	5	72	767	200
8	7	300	9380	2659

Table 3.2: LP outcomes for different values of n on the linear program with min objective function.

Proposition 2 (Fractional Cliques in $\min \mathcal{LP}(G)$). *Let G be a graph with $n \geq 3$ and let $C \cong K_k$ be a maximal complete subgraph of G . Let x be a feasible solution of $\min \mathcal{LP}(G)$. For every $i, j \in V_G$ with $ij \in E_G$*

$$x_{ij} = \frac{1}{k-1}.$$

Proof. Since $C \cong K_n$ is vertex-transitive, the feasible region and the objective function Equation (3.4) are invariant under every permutation of the vertices of G . Hence, if x is optimal, then averaging x over all permutations of the vertex set V produces another optimal solution $\bar{x}$ with

$$\bar{x}_{ij} = w \quad \text{for all } i \neq j \in V$$

for some constant $w \geq 0$. Consider any vertex $i \in V$, where in the clique every pair (i, j) with $j \neq i$ is present, so that $g_{ij} = 1$. By the indicator constraints Equations (2.14)–(2.15),

$$z_{ij} \geq g_{ij} = 1 \quad \text{for all } j \neq i.$$

Together with Equation (2.13) and the product constraints Equations (2.16)–(2.18), this forces each vertex i to carry at least one unit of total incident x -mass on the clique,

$$\sum_{j \in V_C \setminus \{i\}} x_{ij} \geq 1.$$

For the symmetric solution $\bar{x}$, this becomes $(n-1)w \geq 1$, hence $w \geq \frac{1}{n-1}$. On the other hand,

the objective function is $\sum_{i < j, i, j \in V_C} x_{ij} = \binom{n}{2} w$, which is strictly increasing in w . Therefore the

minimum is attained at the smallest feasible value of $w = \frac{1}{n-1}$ which every edge of K_n achieves in the minimizing LP. $\square$

3.3 Max Linear Program

Consider the addition of an objective function maximizing the total weights on all edges to the linear relaxation defined by Equations (2.12)–(3.3) in Section 3.1

$$\max \sum_{i=1}^n \sum_{j=1}^n x_{ij}. \tag{3.5}$$

The construction of the linear program on the arbitrary graph G using Equations (3.1)–(3.3), Equations (2.12)–(2.22) together with Equation (3.5) is denoted as $\max \mathcal{LP}(G)$.

This objective encourages solutions that have fewer fractional edges and therefore tends to produce relaxations that are integral and hence are more informative. In particular, maximizing the total edge weight pushes the variables toward values closer to 1, which promotes direct edges and reduces the need for fractional paths of length 2 to satisfy Equation (2.13). As a result, the relaxation is more likely to yield integral solutions, as illustrated in Table 3.3.

This choice of objective is also useful when identifying square root solutions beyond those unique up to isomorphism. For example, the complete graph K_3 has four roots: itself and three path graphs of length two, which are isomorphic to one another. Figure (3.3) illustrates that the maximization objective preserves consistency up to isomorphism while still distinguishing structurally meaningful solutions.

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	11	0	20	3
6	24	4	107	21
7	43	34	767	200
8	100	207	9380	2659

Table 3.3: LP outcomes for different values of n on the linear program with max objective function.

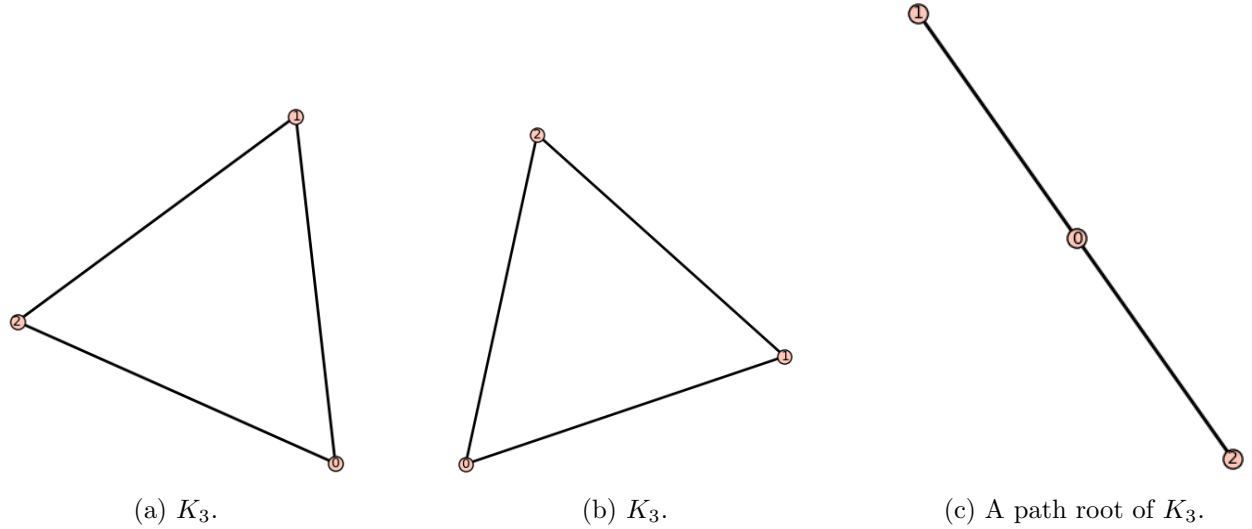


Figure 3.3: The graph K_3 and two of its square roots.

3.3.1 Fundamental Properties

We now establish several fundamental properties of feasible solutions to $\mathcal{LP}(G)$ that follow directly from the formulation. These properties will be used throughout the subsequent analysis of the linear relaxation.

Lemma 12. *Let G and H be graphs such that $G = H^2$. Let $a, b \in V_G$ such that $ab \notin E_G$. For every feasible solution x of $\mathcal{LP}(G)$ and every $v \in V_G$ with $v \neq a, b$ we have $x_{ab} = y_{avb} = 0$.*

Proof. Using Equations (2.14)–(2.15) if $g_{ab} = 0$ then $z_{ab} \leq 0$ and $z_{ab} \geq 0$ such that $z_{ab} = 0$. Then Equation (2.13) gives that $0 = \sum_{v=1}^n y_{avb} + x_{ab}$. Since each x, y variable are non-negative, this only holds when each variable is equal to 0 which gives that $x_{ab} = 0$ and $y_{avb} = 0$ for all vertices $v \in V \setminus \{a, b\}$. $\square$

Lemma 13. *Let G and H be graphs such that $G = H^2$. Let $a, b \in V_G$ such that $ab \notin E_G$. For every feasible solution x of $\mathcal{LP}(G)$ and every $v \in V_G$ with $v \neq a, b$ we have*

$$x_{av} + x_{vb} \leq 1.$$

Proof. Since Lemma (12) gives $y_{avb} = 0$, using Equation (2.15) gives that $0 = y_{avb} \geq x_{av} + x_{vb} - 1$ which after rearranging gives $x_{av} + x_{vb} \leq 1$ as required. $\square$

Lemma 14. *Let G and H be graphs such that $G = H^2$. Let $a, b, v \in V_G$ such that $ab \notin E_G$. If a feasible solution x of $\mathcal{LP}(G)$ satisfies $x_{av} = 1$, then $x_{vb} = 0$.*

Proof. Since $ab \notin E_G$, Proposition (13) gives $x_{av} + x_{vb} \leq 1$. Since $x_{av} = 1$, it follows that $1 + x_{vb} \leq 1$, and hence $x_{vb} \leq 0$. By the non-negativity of the LP variable Equation (3.1), $x_{vb} \geq 0$, and therefore $x_{vb} = 0$. $\square$

Lemma 15 (0 Path Propagation). *Let G and H be graphs such that $G = H^2$. Let $a, i, j, b \in V_G$ such that a, i, j, b form the shortest induced path in H between a and b . For every feasible solution x of $\mathcal{LP}(G)$,*

$$x_{ai} = 1 \implies x_{ib} = 0, \quad x_{ia} = 1 \implies x_{bi} = 0, \quad x_{jb} = 1 \implies x_{aj} = 0, \quad x_{bj} = 1 \implies x_{ja} = 0.$$

Proof. Since a, i, j, b form a path in H , we have $d_H(a, b) = 3$ and therefore $ab \notin E_G$. By Lemma (14), setting $x_{ai} = 1$ gives $x_{ib} = 0$. Similarly, setting $x_{jb} = 1$ gives $x_{aj} = 0$. The remaining two implications follow by symmetry. $\square$

Lemma 16 (1 Path Propagation). *Let G and H be graphs such that $G = H^2$. Let $a, b \in V_G$ such that $ab \in E_G$. If a feasible solution x of $\mathcal{LP}(G)$ satisfies $x_{ab} = 0$, then*

$$\sum_{\substack{k \in V_G \\ g_{ak} > 0, g_{kb} > 0 \\ x_{ak} > 0, x_{kb} > 0}} y_{akb} \geq 1,$$

Proof. Since $ab \in E_G$, we have $g_{ab} = 1$. Hence, by Equation (2.15), $z_{ab} \geq 1$. By Equation (2.13) and the assumption that $x_{ab} = 0$,

$$z_{ab} = \sum_{k=1}^n y_{akb} + x_{ab} = \sum_{k=1}^n y_{akb} \geq 1.$$

It remains to restrict the sum to the stated vertices. If $g_{ak} = 0$, then Equations (2.14) and (2.15) imply $z_{ak} = 0$. By Equation (2.13),

$$0 = z_{ak} = \sum_{r=1}^n y_{ark} + x_{ak},$$

and since all variables are non-negative, $x_{ak} = 0$. Similarly, $g_{kb} = 0$ implies $x_{kb} = 0$. Moreover, if $x_{ak} = 0$, then Equation (2.16) gives $y_{akb} = 0$, while if $x_{kb} = 0$, Equation (2.17) gives $y_{akb} = 0$. Thus every term y_{akb} excluded from the restricted sum is zero, giving

$$\sum_{\substack{k \in V_G \\ g_{ak} > 0, g_{kb} > 0 \\ x_{ak} > 0, x_{kb} > 0}} y_{akb} \geq 1,$$

$\square$

3.4 Leaf Fixing

There are two important structures in a tree.

Definition 3.4.1 (Pendant). A pendant subgraph is the induced subgraph on three vertices s, u, v where s is a leaf with unique neighbour u , and u is a stem of degree 2 with neighbours s and v , where v connects the subgraph to the remainder of the tree.

Definition 3.4.2 (Pendant Fork). A pendant fork subgraph is the induced subgraph on four vertices s_1, s_2, u, v where s_1, s_2 are leaves with unique neighbour u , and u is an internal vertex of degree ≥ 3 where $\{s_1, s_2, v\} \subseteq N(u)$, where v connects the subgraph to the remainder of the tree.

Lemma 17. Every tree contains either a pendant or a pendant fork subgraph.

Proof. Let T be a finite tree with $|V_T| \geq 3$, and root T at an arbitrary vertex. Since T is finite, there exists a leaf s of maximal depth where its parent is u . Every child of u is a leaf, since if there was some child w of u that is not a leaf, then w would have a child and hence there would exist a vertex at depth strictly greater than that of s , contradicting the choice of s .

If u is not the root, let v denote its parent such that every neighbour of u other than v is a leaf.

If u has exactly one leaf neighbour s besides v (when u is not the root), then u has exactly two neighbours, namely s and v , hence $\deg_T(u) = 2$. Therefore the induced subgraph on

$$\{s, u, v\}$$

forms a pendant subgraph.

Otherwise u has at least two leaf neighbours s_1, s_2 . Choose another neighbour v of u distinct from s_1, s_2 ; if u is not the root, take its parent, while if u is the root choose any remaining neighbour. Then

$$\{s_1, s_2, v\} \subseteq N(u),$$

and $\deg_T(u) \geq 3$. Hence the induced subgraph on

$$\{s_1, s_2, u, v\}$$

forms a pendant fork subgraph. □

Using this lemma, every tree contains either a pendant or a pendant fork subgraph. Consequently, whenever G admits a tree root, at least one of these two configurations can be identified and used to derive valid fixings for the variables x_{ij} .

3.4.1 Single Triangular Decomposition

For a vertex $s \in V$, define

$$T(s) = \{\{u, v\} \subseteq V \setminus \{s\} \mid (s, u) \in E, (s, v) \in E, (u, v) \in E\}$$

to be the set of K_3 s or triangles in G that contain s . If $|T(s)| = 1$ where $T(s) = \{\{u, v\}\}$ then the vertex s exists in a unique triangle of G . When the neighbours have different degree $\deg_G(u) \neq \deg_G(v)$, then the vertex with smaller degree must be the stem vertex in the square root. This is assigned deterministically for a single triangle s using

$$x_{sa} = x_{as} = x_{ab} = x_{ba} = 1, \quad \text{where } (a, b) = \begin{cases} (u, v), & \deg_G(u) \leq \deg_G(v), \\ (v, u), & \deg_G(u) > \deg_G(v). \end{cases} \quad (3.6)$$

If $\deg_G(u) = \deg_G(v)$, then either u, v can be chosen since they are connected to a clique.

Proposition 3 (Single Triangle Decomposition Interchangeability). *Let s, u, v form a triangle in square G where s only lies in a single clique. If $\deg_G(u) = \deg_G(v) = d \geq 2$, then one can choose to assign*

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1$$

in the tree root H without loss of generality.

Proof. Since s lies in a single clique in G , in any tree root H , the vertex s must be a leaf, with exactly one of u or v being the internal vertex adjacent to the remainder of the graph. Assume without loss of generality that u is this internal vertex. Since v has no other connections in H besides s and u , any vertex adjacent to v in G other than s must also be adjacent to u in G . Hence

$$N_G(v) \setminus \{s\} \subseteq N_G(u) \setminus \{s\}.$$

Since $\deg_G(u) = \deg_G(v)$, the two sets have equal cardinality, and therefore

$$N_G(u) \setminus \{s\} = N_G(v) \setminus \{s\}.$$

Thus u and v have identical neighbourhoods outside s and may be interchanged without changing the square graph G . Consequently, we may choose u as the internal vertex and assign

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1$$

without loss of generality. □

3.4.2 Tetrahedron Decomposition

A natural extension of Section 3.4.1 can be achieved by considering cliques of size greater than three in order to categorize tree structure whereby an internal vertex is connected to more than one leaf. Whilst finding a clique of size $\geq k$ for fixed k is polynomial, with complexity of order $O(n^k)$ by simply checking all tuples of size k , the decision problem of whether a graph G contains a clique of size k , as well as the problem of finding a largest clique, is NP-complete. Thus, it is necessary to only consider cliques of fixed size 4 which have polynomial complexity $O(n^4)$. This however is sufficient for classifying any pendant fork, since any branching vertex with exactly two leaves will generate a K_4 in the square, whilst a branching vertex with more than two leaves will generate multiple K_4 s of which one can be chosen by arbitrarily selecting any two leaves.

Consider vertices $s_1, s_2, u, v \in V$ such that $K_4 = \{s_1, s_2, u, v\}$ induces a K_4 in G and for each leaf s_1, s_2 of u their neighbourhoods are equivalent and give the common neighbourhood $N_L = N_G(s_1) \setminus K_4 = N_G(s_2) \setminus K_4$. If

$$(N_G(s_1) \setminus K_4) = (N_G(s_2) \setminus K_4) \subset (N_G(u) \setminus K_4) \subseteq (N_G(v) \setminus K_4), \quad (3.7)$$

and if every vertex in the common neighbourhood of each leaf has no neighbours outside the tetrahedron and that neighbourhood itself,

$$N_G(w) \subseteq K_4 \cup N_L \quad \forall w \in N_L, \quad (3.8)$$

then u is designated as the internal vertex, v as the external vertex, and s_1, s_2 as the leaf vertices of the tetrahedron.

Under Equations (3.7) and (3.8), the vertices s_1 and s_2 correspond to leaves adjacent to a common internal vertex u , whilst v acts as the connecting vertex between u and the remainder of the

graph. The vertices with identical minimal neighbourhood structure correspond to leaves adjacent to a common internal vertex u , whilst the vertex v , having maximal neighbourhood containment, acts as the connecting vertex between u and the remainder of the graph.

For $n = 4$, the vertex u is a binary branching vertex contained in a unique K_4 . In particular, when $n = 5$, the branching vertex u has three leaf children s_1, s_2, s_3 , producing $\binom{3}{2} = 3$ distinct K_4 s

$$\{s_1, s_2, u, v\}, \quad \{s_1, s_3, u, v\}, \quad \{s_2, s_3, u, v\}.$$

More generally, suppose u has k leaf children $s_1, \dots, s_k$, and that these vertices together with v induce a K_{k+2} . Then every pair of leaves (s_i, s_j) determines a distinct $K_4 \{s_i, s_j, u, v\}$ where the total number of such K_4 s generated by this branching configuration is $\binom{k}{2}$. Suppose that s_1, s_2 are chosen without loss of generality, then if the K_4 satisfies Equation (3.7) and Equation (3.8) have

$$x_{s_1 u} = x_{us_1} = x_{s_2 u} = x_{us_2} = x_{uv} = x_{vu} = 1. \quad (3.9)$$

Proposition 4 (Tetrahedron Decomposition Interchangeability). *Let s_1, s_2, u, v form a K_4 in square G . If Equation (3.7) and Equation (3.8) are satisfied and*

$$N_G(u) \setminus K_4 = N_G(v) \setminus K_4,$$

then one can choose to assign

$$x_{s_1 u} = x_{us_1} = x_{s_2 u} = x_{us_2} = x_{uv} = x_{vu} = 1$$

in the tree root H without loss of generality.

Proof. Since s_1, s_2, u, v form a K_4 in G and

$$N_G(u) \setminus K_4 = N_G(v) \setminus K_4,$$

the vertices u and v have identical neighbourhoods outside K_4 . Moreover, since u and v are both adjacent to every vertex of K_4 , it follows that

$$N_G(u) = N_G(v).$$

Thus, exchanging u and v is an automorphism of G . Hence, if a tree root H has v as the internal vertex of the tetrahedral configuration, applying this automorphism allows the roles of u and v to be exchanged without changing the square graph. Therefore, without loss of generality may assume that u is the internal vertex of the tetrahedral configuration and assign

$$x_{s_1 u} = x_{us_1} = x_{s_2 u} = x_{us_2} = x_{uv} = x_{vu} = 1.$$

□

3.5 Uniclqual Vertex Linear Program

Using, Lemma (17), the formulation done in Subsection 3.4.1–3.4.2 can be combined into one succinct linear program by first identifying a leaf-stem structure involving three vertices s, u, v . If s is a leaf, u is its corresponding stem, and there exists $v \in N_G[u] \setminus \{u\}$ such that

$$N_G[s] \subset N_G[u] \subseteq N_G[v],$$

with all vertices in $N_G[s] \setminus \{u, v\}$ having the same closed neighbourhood as s , then impose the fixing

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1. \quad (3.10)$$

Proposition 5 (Unicliqual Vertex Interchangeability). *Let s, u, v form a triangle in square G , where s is unicliqual. If $N_G[u] = N_G[v]$ then u and v may be chosen arbitrarily as the corresponding vertex in the tree root H without loss of generality.*

Proof. Since s is unicliqual, the maximal clique containing s is uniquely determined. If this clique satisfies the conditions of Proposition 3, then that proposition permits either u or v to be chosen as the corresponding vertex, since $N_G[u] = N_G[v]$ gives $\deg_G(u) = \deg_G(v)$. If the configuration instead satisfies the conditions of Proposition 4, then the equality $N_G[u] = N_G[v]$ gives

$$N_G(u) \setminus K_4 = N_G(v) \setminus K_4,$$

so either u or v may be chosen in the tetrahedron fixing. Hence, in either case, u and v are interchangeable, and either may be chosen without loss of generality. $\square$

It remains to verify that whenever a root H of G exists, the addition of the constraint Equation (3.10) preserves the feasibility of the linear program.

Proposition 6 (Validity of Unicliqual Vertex Fix). *If a root H of G exists, then $\max \mathcal{UVLP}(G)$ is feasible.*

Proof. Suppose that a root H of G exists. By Proposition 1, $\mathcal{LP}(G)$ is feasible. It remains to show that there exists a root whose adjacency matrix satisfies Equation (3.10).

Let s, u, v be the vertices identified by the unicliqual vertex fixing. By Proposition 5, if s, u, v form the corresponding triangle and $N_G[u] = N_G[v]$, then u and v may be chosen arbitrarily as the corresponding vertex in the tree root without loss of generality. Thus, if the given root H does not satisfy Equation (3.10), the roles of u and v may be interchanged to obtain another root H' of G for which

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1.$$

Hence the adjacency matrix of H' is a feasible solution of $\mathcal{IP}(G)$ that satisfies Equation (3.10). Since every feasible solution of $\mathcal{IP}(G)$ is feasible for $\mathcal{LP}(G)$, this solution is also feasible for $\max \mathcal{UVLP}(G)$. Therefore, $\max \mathcal{UVLP}(G)$ is feasible whenever a root H of G exists. $\square$

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	11	0	21	2
6	25	3	117	11
7	55	22	852	115
8	154	153	10271	1768

Table 3.4: Outcomes of the unicliqual vertex linear program for graphs of different orders of n .

3.6 Tree Proof

We first show that the unicliqual vertex linear program will always fix a pendant or pendant fork.

Lemma 18. *Let G and T be graphs such that $G = T^2$ and T is a tree. $\max \mathcal{UVLP}(G)$ always a pendant or pendant fork subgraph of T .*

Proof. By Lemma 17, T either contains pendant or a pendant fork subgraph. Let s be a leaf of this subgraph and let u be its stem. Since s is a leaf of T , every vertex of $N_G[s]$ is either s , u , the neighbours of u in T , or a vertex at distance two from s through u . Hence

$$N_G[s] \subset N_G[u].$$

Moreover, if v is the neighbour of u in T that connects the pendant or pendant fork to the remainder of T , then every vertex adjacent to u in T is also within distance two of v in T . This gives

$$N_G[u] \subseteq N_G[v].$$

In the pendant fork case, the other leaf s' of the fork satisfies $N_G[s'] = N_G[s]$, since both s and s' are leaves adjacent to the same vertex u . Thus the vertices in $N_G[s] \setminus \{u, v\}$ have the same closed neighbourhood as s .

Therefore the vertices s, u, v satisfy the conditions for Equation (3.10), which imposes

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1.$$

□

Lemma 19 (Propagation). *Let G and T be graphs such that $G = T^2$ and T is a tree. Let $s, u, v \in V_G$ be identified such that s is a leaf, u is its stem, and v is the external vertex, with*

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1.$$

Let $N_T(v) \setminus u = \{a_1, \dots, a_r\}$ be the set of all other neighbours of v in T . Then path propagation fixes

$$x_{va_i} = x_{a_iv} = 1 \text{ for every } i \in \{1, \dots\}.$$

Proof. Since s is a leaf and u is a stem of T , the path $s-u-v$ is contained in T . Let $A = \{a_1, \dots, a_r\}$ denote the vertices adjacent to v in T other than u , and let $B = \{b_1, \dots, b_s\}$ denote the vertices adjacent to a vertex in A other than v . For each $a_i \in A$, the vertices s, u, v, a_i form a P_4 in T . Since $x_{su} = 1$ and there exists no cycles in T , Lemma (15) gives

$$x_{ua_i} = 0 \quad \text{for all } a_i \in A.$$

Without loss of generality consider $a_1 \in A$. Since $ua_1 \in E_G$, have $g_{ua_1} = 1$, and hence Equation (2.15) gives $z_{ua_1} \geq 1$. Since $x_{ua_1} = 0$, Equation (2.13) gives

$$1 \leq z_{ua_1} = x_{ua_1} + \sum_{k \in V_G} y_{uka_1} = \sum_{k \in V_G} y_{uka_1}.$$

The terms corresponding to $k = s$ and $k = u$ are zero since $g_{sa_1} = g_{uu} = 0$, and hence Lemma (12) gives $x_{sa_1} = x_{uu} = 0$. Therefore, by Equations (2.16)–(2.18),

$$y_{usa_1} = y_{uua_1} = 0.$$

For every $m \in A$, we have $x_{um} = 0$, and hence Equation (2.16) gives

$$y_{uma_1} = 0.$$

Similarly, for every $n \in B$, we have $g_{un} = 0$, and therefore Lemma (12) gives $x_{un} = 0$. Thus Equation (2.16) gives

$$y_{una_1} = 0.$$

Thus all terms in the sum are zero except possibly y_{uva_1} , giving

$$1 \leq y_{uva_1}.$$

Since $x_{uv} = 1$, Equation (2.16) gives $y_{uva_1} \leq x_{uv} = 1$, while Equation (2.17) gives $y_{uva_1} \leq x_{va_1}$. Hence $1 \leq y_{uva_1} \leq x_{va_1} \leq 1$, and therefore

$$x_{va_1} = 1.$$

This process can be repeated for each $a_i \in A$, giving $x_{va_i} = 1$ for all $i \in \{1, \dots, r\}$. Thus the edges $uv, va_1, \dots, va_r$ are identified in the root, with u acting as a leaf, v as its stem, and each a_i as the next vertex along the tree. $\square$

Theorem 6 (Inductive Proof). *Let G and T be graphs such that $G = T^2$ and T is a tree. Then $\max \mathcal{UVLP}(G)$ returns an integral solution.*

Proof. Suppose that $G = T^2$ for some tree T . By Lemma 18, the uniclqual vertex fixing identifies a leaf s , its stem u , and the external vertex v , and Equation (3.10) imposes

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1.$$

Hence the hypotheses of Lemma 19 are satisfied.

We prove by induction on the number of edges of the reconstructed subtree that repeated application of path propagation assigns an integral value to every edge of T .

For the base case, the reconstructed subtree consists of the edges initially identified by the uniclqual vertex fixing. These edges have integral assignment by Equation (3.10).

For the inductive step, suppose that a connected subtree T' of T containing the fixed path $s - u - v$ has been reconstructed and that every edge of T' has been assigned its integral value. If $T' \neq T$, then, since T is connected, there exists an edge

$$ab \in E_T \setminus E_{T'}$$

such that $a \in V_{T'}$ and $b \notin V_{T'}$.

Since T is a tree, the edge ab extends T' without creating a cycle. Moreover, the unique path in T from the fixed path $s - u - v$ to b consists of edges already reconstructed up to a , followed by the edge ab . Thus path propagation can be applied to this extended path, and fixes

$$x_{ab} = x_{ba} = 1.$$

Consequently, T' can be extended to the larger connected subtree $T'' = T' + ab$, whose edges all have integral assignments.

Therefore, whenever the reconstructed subtree is a proper subtree of T , path propagation identifies at least one additional edge of T . Since T is finite, repeated application of this argument eventually reconstructs all of T and assigns an integral value to every edge.

Hence the resulting solution is integral and corresponds to the tree root T of G . Therefore $\max \mathcal{UVLP}(G)$ returns an integral solution. $\square$

Theorem 7 (Path Proof). *Let G and T be graphs such that $G = T^2$ and T is a tree. Then $\max \mathcal{UVLP}(G)$ returns an integral solution.*

Proof. Suppose that $G = T^2$ for some tree T . By Lemma 18, the uniclinal vertex fixing identifies a leaf s , its stem u , and the external vertex v , and Equation (3.10) imposes

$$x_{su} = x_{us} = x_{uv} = x_{vu} = 1.$$

Hence the hypotheses of Lemma 19 are satisfied.

In particular letting $N_T(v) \setminus \{u\} = \{a_1, \dots, a_r\}$ Lemma 19 fixes

$$x_{va_i} = x_{a_i v} = 1 \text{ for every } i \in \{1, \dots, r\}.$$

The same argument can then be applied at each newly identified vertex. More generally, let $w \in V(T)$ be arbitrary. Since T is connected, there is a path from v to w . Therefore, together with the fixed path $s - u - v$, there is a path

$$s - u - v - \dots - w$$

in T . Starting from the fixed edges su and uv , path propagation can be applied successively along this path. Hence every edge on the path from v to w is assigned an integral value.

Since w was arbitrary, every vertex of T is reached by such a path. Now let $ab \in E_T$ be arbitrary. Since T is connected, there is a path from v to a , and appending the edge ab gives a path

$$s - u - v - \dots - a - b$$

containing ab . Thus path propagation assigns an integral value to ab .

Therefore every edge of T is assigned an integral value. Hence the resulting assignment is integral and corresponds to the tree root T of G . Therefore $\max \mathcal{UVLP}(G)$ returns an integral solution. $\square$

Chapter 4

Further Work

In this chapter, we discuss additional work, further concepts, and interesting ideas arising from structural properties that the square graph intrinsically possesses. Most of these ideas have not been rigorously proven and are instead presented through proof sketches or intuition for why they may help improve the results obtained by the linear programming framework.

Previous methods for solving specific classes of roots have been developed by augmenting the linear programming framework with additional constraints and objective functions. Nevertheless, integrating known polynomial-time solutions for trees into an overall framework, while maintaining the internal consistency that valid square graphs must always correspond to feasible instances, does not appear to justify the use of a comparatively complex polynomial-time framework such as the interior-point method.

It is, however, relatively straightforward to construct stronger exact algorithms by performing a polynomial number of runs of the linear program. Each run can be used to identify additional structural information, fix variables, or further restrict the possible roots before solving the resulting instance again. This provides a useful mechanism through which improvements to the base linear program can have a much greater effect than the improvement obtained from a single LP solution alone. In particular, a stronger base program can provide more integral variables and structural information, which can then be propagated through subsequent runs of the program.

The following sections illustrate several directions in which this can be explored. Rather than treating each structural property as an isolated addition to the formulation, these examples demonstrate how improvements to the base linear program can be repeatedly exploited to obtain substantially stronger results.

4.1 Vertex Deletion

From Table (3.4), we can already see that for small n , all square graphs can be completely categorized such that, whenever a root H exists, an integral solution is obtained, while whenever no root H exists, the linear program is infeasible. This suggests that the behaviour of the linear program may be closely related not only to the structure of the graph itself, but also to the structure of its subgraphs.

In particular, consider arbitrarily removing a vertex from G and applying the same linear program to the resulting subgraph. If the subgraph has an integral solution, this provides structural information about a possible root on the reduced vertex set. The natural question is whether this information can subsequently be used to constrain the original graph when the deleted vertex is reintroduced.

More specifically, if deleting a vertex produces a graph for which the linear program has an integral solution, it is worth investigating whether the corresponding solution can be extended to the original graph and, if so, whether the possible extensions can be restricted to a small number of cases. If such extensions can be characterized, vertex deletion provides a mechanism through which information obtained from smaller instances can be propagated back to the original graph.

Let G^- be the subgraph of G defined by $(V^- = V - v, E_{G^-} = E_G \setminus uv \mid u \in V^-)$. If G^- has a valid square root, denote it by H^- . Let H be a square root of G satisfying $H^2 = G$, and let H_- be the subgraph of H obtained by deleting v , defined by $(V^-, E_{H_-} \setminus uv \mid u \in V^-)$.

In general, H_- need not be a square root of G^- , since the deletion of v from H may remove P_2 s between vertices in V^- which results in $(H_-)^2 \neq G^-$. Nevertheless, an integral solution of the LP for G^- provides a candidate set of root edges on V^- which can be used to refine the LP for G .

Algorithm 2 Vertex-Deletion Linear Program

```

1: Input: A graph  $G$ 
2: Output: LP model  $(m, x)$ 
3: Construct and solve  $\mathcal{LP}(G)$ .
4: if  $\mathcal{LP}(G)$  is integral then
5:   return  $(m, x)$ 
6: end if
7: Set  $F = \left\{ \{a, b\} \in \binom{V_G}{2} : x_{ab} = x_{ba} = 1 \right\}$ .
8: Compute  $d_F(u) = |\{w \in N_G(u) : \{u, w\} \notin F\}|$  for each  $u \in V_G$ .
9: Choose  $v \in \arg \min_{u \in V_G} d_F(u)$  and set  $V^- = V_G \setminus \{v\}$ .
10: Construct  $G^- = G[V^-]$  and inherit the fixing  $F^- = \{\{a, b\} \in F : a, b \in V^-\}$ .
11: Construct and solve  $\mathcal{LP}(G^-) + F^-$ .
12: if  $\mathcal{LP}(G^-) + F^-$  is integral then
13:   Construct  $H^-$  from its integral solution.
14: else
15:   return  $(m, x)$ 
16: end if
17: Construct a new copy of  $\mathcal{LP}(G)$  and impose  $x_{ab} = x_{ba} = 1$  for every  $\{a, b\} \in F$  and every
     $ab \in E_{H^-}$ .
18: Solve the modified LP.
19: if the modified LP is infeasible then
20:   return  $(m, x)$ 
21: else
22:   return the modified LP solution  $(m, x)$ 
23: end if

```

The recursive algorithm works as follows.

- i) Solve $\mathcal{LP}(G)$. If the LP is infeasible, the procedure terminates. If the LP is integral, the corresponding graph is retained as a candidate square root and the procedure terminates. Otherwise, let $F \subseteq \binom{V}{2}$ denote the set of unordered vertex pairs for which the solution satisfies $x_{ab} = x_{ba} = 1$.
- ii) For each $u \in V$, define its unfixed degree with respect to F by $d_F(u) = |\{w \in N_G(u) : \{u, w\} \notin F\}|$. Choose a vertex $v \in \arg \min_{u \in V} d_F(u)$ and construct $G^- = G - v$ by deleting v and all edges

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	11	0	21	2
6	25	3	117	11
7	62	15	852	115
8	175	132	10271	1768

Table 4.1: Outcomes of the vertex deletion linear program using uniclqual vertex as the base for graphs of different orders of n .

incident to v . Let $V^- = V \setminus v$ and let $F^- = a, b \in F, b \in V^-$ denote the fixing inherited by G^- .

- iii) Solve $\mathcal{LP}(G^-) + F^-$, which is G^- with the additional fixing F^- .
 - (a) If $\mathcal{LP}(G^-) + F^-$ is fractional, let H^- denote the corresponding root of F^- .
 - (b) If $\mathcal{LP}(G^-) + F^-$ is infeasible, the candidate H^- is discarded.
 - (c) If $\mathcal{LP}(G^-) + F^-$ is fractional, recursively apply the vertex deletion algorithm to solve $\mathcal{G}^-$. If the recursive deletion produces an integral solution, let H^- denote the corresponding root of G^- .
- iv) If an integral candidate H^- has been obtained, construct $\mathcal{LP}(G \mid F, H^-)$ by taking $\mathcal{LP}(G)$ and imposing $x_{ab} = x_{ba} = 1$ for every $a, b \in F$ and every pair $a, b \in \binom{V^-}{2}$ such that $ab \in E_{H^-}$. No additional constraint is imposed on pairs for which $ab \notin E_{H^-}$, and the variables corresponding to pairs involving v remain unrestricted apart from the fixing induced by F .
- v) Solve $\mathcal{LP}(G \mid F, H^-)$.
 - (a) If $\mathcal{LP}(G \mid F, H^-)$ is feasible, the fixing obtained from H^- is consistent with the LP formulation for G , and the resulting solution is retained.
 - (b) If $\mathcal{LP}(G \mid F, H^-)$ is infeasible, the candidate H^- is discarded. This does not imply that $\mathcal{LP}(G)$ is infeasible; it only shows that the candidate root structure H^- is incompatible with the current fixing for G .
- vi) If no integral candidate H^- is obtained from $\mathcal{LP}(G^-) + F^-$ or from the recursive application to G^- , then no further fixing is obtained from the vertex deletion step. The current feasible solution of $\mathcal{LP}(G)$ is retained, and the procedure terminates.
- vii) If an integral candidate H^- is obtained but $\mathcal{LP}(G \mid F, H^-)$ is infeasible, the fixing induced by H^- is discarded. The current feasible solution of $\mathcal{LP}(G)$ is retained, and the procedure terminates. This does not imply that $\mathcal{LP}(G)$ is infeasible; it only shows that the candidate root structure H^- is incompatible with the current fixing.

If the candidate H^- is discarded, instead of applying Step vi), consider the remaining square roots of G^- , taking one representative from each isomorphism class. For each such root H' , construct $\mathcal{LP}(G \mid F, H')$ by imposing $x_{ab} = x_{ba} = 1$ for every $a, b \in \binom{V^-}{2}$ such that $ab \in E_{H'}$, in

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	11	0	21	2
6	25	3	117	11
7	65	12	852	115
8	199	108	10271	1768

Table 4.2: Outcomes of the recursive vertex deletion linear program using uniclqual vertex as the base for graphs of different orders of n .

addition to the fixing induced by F . If the resulting LP is feasible, the corresponding solution is retained.

The intuition behind this step is that the linear program only finds a solution up to isomorphism, while there may be many distinct solutions that are non-isomorphic. Hence, although fixing a subgraph obtained from the LP can often be useful, the particular subgraph generated by the LP may not correspond to the required subgraph of a square root. If finding an appropriate isomorphism of a known root could be done in polynomial time, then such an algorithm would be polynomial. However, the current implementation does not provide this, and is therefore not polynomial. Nevertheless, the approach shows very strong improvements in practice.

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	11	0	21	2
6	28	0	117	11
7	72	5	852	115
8	254	53	10271	1768

Table 4.3: Outcomes of the vertex deletion linear program using uniclqual vertex as the base and enumerating all possible H^- roots for graphs of different orders of n .

n	H exists		H does not exist	
	LP integral solution	LP fractional solution	LP infeasible	LP fractional solution
3	3	0	1	0
4	6	0	5	0
5	11	0	21	2
6	28	0	117	11
7	74	3	852	115
8	256	51	10271	1768

Table 4.4: Outcomes of the recursive vertex deletion linear program using uniclqual vertex as the base and enumerating all possible H^- roots for graphs of different orders of n .

4.2 Symbolic Groups

Lemma 13 give extremely strong structure into how the linear program operates. Using the maximization objective function Lemma 13, any two edges ij , jk which exist in G but ki , can be assigned the weights a , b respectively where $x_{ij} = a$, $x_{jk} = b$ and $a + b \leq 1$.

If under certain conditions the relationship holds with equality $a + b = 1$, one would be able to eventually categorize and formulate equations for many of the weights on the various edges. In particular, this is of great important in resolving scenarios where the maximal fractional objective function is greater than the optimal integer objective function corresponding to a valid square root.

Consider this graph square on 6 vertices with three different roots

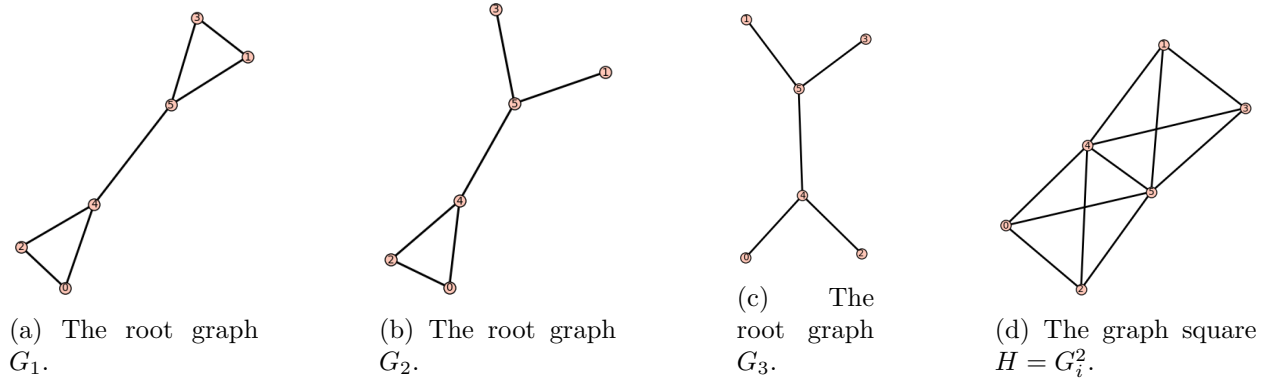


Figure 4.1: Three non-isomorphic square roots of the same six-vertex graph square.

G_1 is the unique maximal root. Assigning $a = x_{14}$ gives using Lemma (??) that $1 - a = x_{40}$ since 01 does not exist in G . The same applies such that $1 - a = x_{42}$ and $a = x_{43}$, whereby the existence of edges 13 and 02 prevent additional constraints being imposed upon a , it is internally consistent. By symmetry can likewise assign $b = x_{15} = x_{35}$ and $1 - b = x_{50} = x_{52}$. Setting $a = 0$ and $b = 1$ thus gives x_{40}

Consider this graph square on 6 vertices with only one unique root

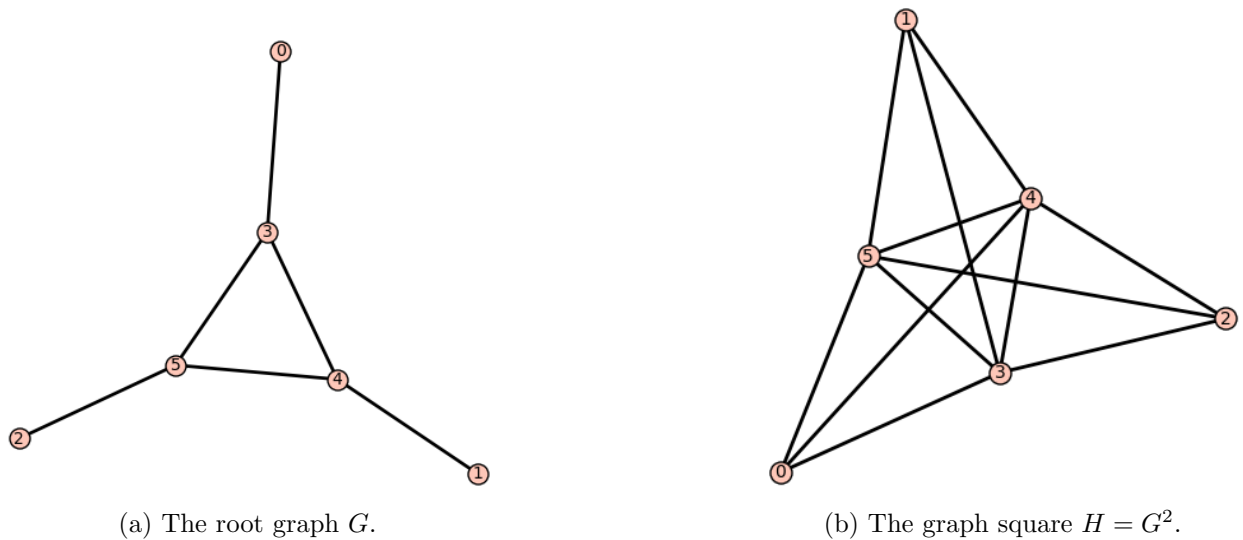


Figure 4.2: A six-vertex graph and its square.

Assigning $a = x_{03}$ gives using Lemma (??) that $x_{31} = 1 - a$ since 01 does not exist in G , and likewise with 12 does not exist in G have $x_{32} = a$. However with the absence of 02, must have $x_{03} + x_{32} \leq 1$ such that $a \leq \frac{1}{2}$. By symmetry this likewise holds for $b = x_{05} = x_{15}$, $1 - b = x_{52}$ and $c = x_{42} = x_{41}$, $1 - c = x_{40}$ giving $b, c \leq \frac{1}{2}$.

In this case, rather than maximizing the objective function and giving a fractional solution, setting $a = b = c = 0$ gives a valid integral solution.

4.3 Duality

One of the strengths of a linear program in solving problems is the ability to leverage duality. The Karush-Kuhn-Tucker (KKT) conditions provide the optimality conditions for a constrained optimization problem. For a linear program, these conditions reduce to primal feasibility, dual feasibility, and complementary slackness. In particular, when both the primal and dual are feasible, strong duality implies that their optimal objective values are equal.

Consider the dual formulation of the max objective linear program defined in Section 3.3 by first restating it in canonical form.

Objective Function:

$$\min - \sum_{i=1}^n \sum_{j=1}^n x_{ij}. \quad (4.1)$$

Variables:

$$x_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.2)$$

$$y_{ikj} \in \mathbb{R}_{\geq 0}, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (4.3)$$

$$z_{ij} \in \mathbb{R}_{\geq 0}, \quad \forall i, j \in \{1, \dots, n\}. \quad (4.4)$$

Constraints:

$$(1) \text{ Indicator argument: } z_{ii} - \sum_{k=1}^n y_{iki} + \sum_{k=1}^n x_{ik} - x_{ii} = 0, \forall i \in \{1, \dots, n\}, \quad (4.5)$$

$$z_{ij} - \sum_{k=1}^n y_{ikj} - x_{ij} = 0, \quad \forall i \neq j \in \{1, \dots, n\}, \quad (4.6)$$

$$(2) \text{ Indicator function: } z_{ij} - M g_{ij} \leq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.7)$$

$$-z_{ij} + g_{ij} \leq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.8)$$

$$(3) \text{ Product: } y_{ikj} - x_{ik} \leq 0, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (4.9)$$

$$y_{ikj} - x_{kj} \leq 0, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (4.10)$$

$$x_{ik} + x_{kj} - y_{ikj} - 1 \leq 0, \quad \forall i, k, j \in \{1, \dots, n\}, \quad (4.11)$$

$$(4) \text{ Symmetry: } x_{ij} - x_{ji} = 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.12)$$

$$(5) \text{ Diagonal: } x_{ii} = 0, \quad \forall i \in \{1, \dots, n\}, \quad (4.13)$$

$$(6) \text{ Variable Bound: } x_{ij} - 1 \leq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (4.14)$$

$$y_{ikj} - 1 \leq 0, \quad \forall i, k, j \in \{1, \dots, n\}. \quad (4.15)$$

Its dual can be rewritten as

Objective Function:

$$\max \sum_{i=1}^n \sum_{j=1}^n \left(-M g_{ij} \gamma_{ij} + g_{ij} \delta_{ij} - \lambda_{ij} \right) - \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n (\eta_{ikj} + \mu_{ikj}). \quad (4.16)$$

Dual Variables:

$$\alpha_i \in \mathbb{R}, \quad \forall i \in \{1, \dots, n\}, \quad (\text{associated with (4.5)}), \quad (4.17)$$

$$\beta_{ij} \in \mathbb{R}, \quad \forall i \neq j \in \{1, \dots, n\}, (\text{associated with (4.6)}), \quad (4.18)$$

$$\gamma_{ij} \geq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (\text{associated with (4.7)}), \quad (4.19)$$

$$\delta_{ij} \geq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (\text{associated with (4.8)}), \quad (4.20)$$

$$\epsilon_{ikj} \geq 0, \quad \forall i, k, j \in \{1, \dots, n\}, (\text{associated with (4.9)}), \quad (4.21)$$

$$\zeta_{ikj} \geq 0, \quad \forall i, k, j \in \{1, \dots, n\}, (\text{associated with (4.10)}), \quad (4.22)$$

$$\eta_{ikj} \geq 0, \quad \forall i, k, j \in \{1, \dots, n\}, (\text{associated with (4.11)}), \quad (4.23)$$

$$\theta_{ij} \in \mathbb{R}, \quad \forall i, j \in \{1, \dots, n\}, \quad (\text{associated with (4.12)}), \quad (4.24)$$

$$\iota_i \in \mathbb{R}, \quad \forall i \in \{1, \dots, n\}, \quad (\text{associated with (4.13)}), \quad (4.25)$$

$$\lambda_{ij} \geq 0, \quad \forall i, j \in \{1, \dots, n\}, \quad (\text{associated with (4.14)}), \quad (4.26)$$

$$\mu_{ikj} \geq 0, \quad \forall i, k, j \in \{1, \dots, n\}, (\text{associated with (4.15)}). \quad (4.27)$$

Dual Constraints:

$\forall i, j \in \{1, \dots, n\}$, each primal x_{ij} :

$$\begin{cases} -1 - \sum_{k=1}^n \epsilon_{iik} - \sum_{k=1}^n \zeta_{kii} + \sum_{k=1}^n \eta_{iik} + \sum_{k=1}^n \eta_{kii} + \iota_i + \lambda_{ii} \geq 0, & \text{if } j = i, \\ -1 + \alpha_i - \beta_{ij} - \sum_{k=1}^n \epsilon_{ikj} - \sum_{k=1}^n \zeta_{kij} + \sum_{k=1}^n \eta_{ikj} + \sum_{k=1}^n \eta_{kij} + \theta_{ij} - \theta_{ji} + \lambda_{ij} \geq 0, & \text{if } j \neq i, \end{cases} \quad (4.28)$$

$\forall i, k, j \in \{1, \dots, n\}$, each primal y_{ikj} :

$$\begin{cases} -\alpha_i + \epsilon_{iki} + \zeta_{iki} - \eta_{iki} + \mu_{iki} \geq 0, & \text{if } j = i, \\ -\beta_{ij} + \epsilon_{ikj} + \zeta_{ikj} - \eta_{ikj} + \mu_{ikj} \geq 0, & \text{if } j \neq i, \end{cases} \quad (4.29)$$

$\forall i, j \in \{1, \dots, n\}$, each primal z_{ij} :

$$\begin{cases} \alpha_i + \gamma_{ii} - \delta_{ii} = 0, & \text{if } j = i, \\ \beta_{ij} + \gamma_{ij} - \delta_{ij} = 0, & \text{if } j \neq i. \end{cases} \quad (4.30)$$

whereby the Lagrangian is

$$\begin{aligned}
\mathcal{L}(x, y, z) = & - \sum_{i=1}^n \sum_{j=1}^n x_{ij} + \sum_{i=1}^n \alpha_i \left(z_{ii} - \sum_{k=1}^n y_{iki} + \sum_{k=1}^n x_{ik} - x_{ii} \right) + \sum_{i \neq j} \beta_{ij} \left(z_{ij} - \sum_{k=1}^n y_{ikj} - x_{ij} \right) \\
& + \sum_{i=1}^n \sum_{j=1}^n \gamma_{ij} (z_{ij} - M g_{ij}) + \sum_{i=1}^n \sum_{j=1}^n \delta_{ij} (-z_{ij} + g_{ij}) + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \epsilon_{ikj} (y_{ikj} - x_{ik}) \\
& + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \zeta_{ikj} (y_{ikj} - x_{kj}) + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \eta_{ikj} (x_{ik} + x_{kj} - y_{ikj} - 1) \\
& + \sum_{i=1}^n \sum_{j=1}^n \theta_{ij} (x_{ij} - x_{ji}) + \sum_{i=1}^n \iota_i x_{ii} + \sum_{i=1}^n \sum_{j=1}^n \lambda_{ij} (x_{ij} - 1) + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \mu_{ikj} (y_{ikj} - 1).
\end{aligned}$$

Collecting terms gives

$$\begin{aligned}
\mathcal{L}(x, y, z) = & \sum_{i=1}^n \sum_{j=1}^n x_{ij} \left[-1 + \mathbf{1}_{\{j=i\}} \left(\alpha_i - \sum_{k=1}^n \epsilon_{iki} - \sum_{k=1}^n \zeta_{kii} + \sum_{k=1}^n \eta_{iki} + \sum_{k=1}^n \eta_{kii} + \iota_i + \lambda_{ii} \right) \right. \\
& + \mathbf{1}_{\{j \neq i\}} \left(\alpha_i - \beta_{ij} - \sum_{k=1}^n \epsilon_{ikj} - \sum_{k=1}^n \zeta_{kij} + \sum_{k=1}^n \eta_{ikj} + \sum_{k=1}^n \eta_{kij} + \lambda_{ij} \right) + \theta_{ij} - \theta_{ji} \Big] \\
& + \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n y_{ikj} \left[-\mathbf{1}_{\{j=i\}} \alpha_i - \mathbf{1}_{\{j \neq i\}} \beta_{ij} + \epsilon_{ikj} + \zeta_{ikj} - \eta_{ikj} + \mu_{ikj} \right] \\
& + \sum_{i=1}^n \sum_{j=1}^n z_{ij} \left[\mathbf{1}_{\{j=i\}} \alpha_i + \mathbf{1}_{\{j \neq i\}} \beta_{ij} + \gamma_{ij} - \delta_{ij} \right] \\
& - \sum_{i=1}^n \sum_{j=1}^n M g_{ij} \gamma_{ij} + \sum_{i=1}^n \sum_{j=1}^n g_{ij} \delta_{ij} - \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \eta_{ikj} - \sum_{i=1}^n \sum_{j=1}^n \lambda_{ij} - \sum_{i=1}^n \sum_{k=1}^n \sum_{j=1}^n \mu_{ikj}.
\end{aligned}$$

In this scenario, the dual is tight and gives exactly the same objective function value as $\max \mathcal{LP}(G)$, consistent with the strong duality theorem. The results in Table 4.5 show that every primal-feasible instance has a feasible dual, including both integral and fractional primal solutions, while the instances for which the primal is infeasible also have an infeasible dual. The KKT conditions, through complementary slackness, further characterize these optimal solutions by identifying which primal and dual constraints are tight. In particular, positive dual variables correspond to tight primal constraints, providing a potential means of identifying the constraints that determine the optimal objective value and of investigating the structure of fractional solutions.

It remains to explore this more by investigating the structure of the optimal dual solutions, the relationship between positive dual variables and fractional primal variables, and whether dual information can be used to derive additional valid constraints for the square-root recognition problem.

4.4 Rounding

Finally, we consider a simple heuristic for further refining fractional LP solutions - rounding. Since each edge variable takes a value in $[0, 1]$, many strategies are possible, ranging from probabilistic rounding to systematically fixing each fractional edge in both possible ways. Because every edge

n	H	Primal	Dual	Count
5	Exists	Integer	Feasible	11
5	Does not exist	Fractional	Feasible	3
5	Does not exist	Infeasible	Infeasible	20
6	Exists	Integer	Feasible	25
6	Exists	Fractional	Feasible	3
6	Does not exist	Fractional	Feasible	21
6	Does not exist	Infeasible	Infeasible	107
7	Exists	Integer	Feasible	44
7	Exists	Fractional	Feasible	33
7	Does not exist	Fractional	Feasible	200
7	Does not exist	Infeasible	Infeasible	767

Table 4.5: Primal and dual LP outcomes for graphs of different orders n .

variable must ultimately be 0 or 1, a more exhaustive approach could branch on each fractional variable to produce a binary search tree.

This yields another polynomial-time algorithm, provided the number of fractional variables is bounded by $O(\log n)$, so that the tree has only polynomially many leaves. Classical techniques such as pruning, bounding, and depth-first or breadth-first search can then be leveraged to evaluate all end nodes efficiently: pruning discards branches whose LP bound exceeds the incumbent, bounding limits the depth of exploration, and the search order determines how quickly an integral solution is found.

In this example, we use only a simple greedy rounding procedure. At each iteration, the current fractional LP solution is examined and a single fractional edge is selected. The selection is based on the distance of its value from 0.5, so that fractional values closer to 0 or 1 are considered first. The selected edge is then fixed to 0 if its value is below 0.5 and to 1 otherwise. The LP is subsequently resolved with all previous fixings retained.

If the resulting LP is infeasible, the rounding procedure terminates and the previous feasible fractional solution is retained. If the resulting LP remains fractional, another fractional edge is selected and the process continues. The procedure terminates when an integral solution is obtained or when no further feasible fixing can be made.

The procedure is greedy because only one fixing is selected at each iteration and no alternative fixing is explored. Thus, unlike exhaustive rounding or branch-and-bound approaches, it does not construct a binary search tree of possible fixings. Since there are at most $\binom{|V_G|}{2}$ edge variables to fix and each iteration requires one LP solve, the rounding procedure requires at most $O(n^2)$ LP solves. Consequently, as $\mathcal{UVLP}(G)$ can be solved in polynomial time, the greedy rounding procedure is polynomial-time.

Rounding fractional solutions in this manner either produces an integral solution or reaches a point at which the selected fixing makes the LP infeasible. These outcomes give the classifications reported in Table 4.6.

Algorithm 3 Greedy LP Rounding

```

1: Input: A graph  $G$ 
2: Output: A LP solution  $(m, x)$ 
3: Initialize  $F = \emptyset$  and construct  $\mathcal{UVLP}(G)$ .
4: Solve  $\mathcal{UVLP}(G)$ .
5: while the current LP solution is fractional do
6:   For each fractional edge  $ab$ , compute its rounding score  $s_{ab} = |x_{ab} - 0.5|$ .
7:   Select a fractional edge  $ab$  with maximum score.
8:   Set

$$\hat{x}_{ab} = \hat{x}_{ba} = \begin{cases} 0, & x_{ab} < 0.5, \\ 1, & x_{ab} \geq 0.5. \end{cases}$$

9:   Add  $ab$  with its assigned value to  $F$ .
10:  Reconstruct  $\mathcal{UVLP}(G)$  with all fixings in  $F$ .
11:  Solve  $\mathcal{UVLP}(G) + F$ .
12:  if  $\mathcal{UVLP}(G) + F$  is infeasible then
13:    return the previous feasible solution  $(m, x)$ .
14:  end if
15:  if  $\mathcal{UVLP}(G) + F$  is integral then
16:    return the integral solution  $(m, x)$ .
17:  end if
18: end while
19: return  $(m, x)$ .

```

n	H exists		H does not exist		Integral $\leftarrow$ Fractional
	LP integral	LP fractional	LP infeasible	LP fractional	
3	3	0	1	0	0
4	6	0	5	0	0
5	11	0	21	2	0
6	27	1	117	11	2
7	75	2	852	115	20
8	262	45	10271	1768	108

Table 4.6: Outcomes of the greedy rounding linear program using the uniclqual vertex linear program as the base for graphs of different orders of n .

Listing 1: Integer Program.

```

1 def integer_program(G):
2     m = MixedIntegerLinearProgram(solver = 'GLPK', maximization=False)
3     m.solver_parameter("simplex_or_intopt", "intopt_only")
4
5     x = m.new_variable(integer=True, nonnegative=True)
6     y = m.new_variable(integer=True, nonnegative=True)
7     z = m.new_variable(integer=True, nonnegative=True)
8
9     A_G = G.adjacency_matrix()
10    n = A_G.nrows()
11    M = n**2
12
13    # Indicator argument
14    for i in range(n):
15        for j in range(n):
16            if i == j:
17                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
18                    - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
19            else:
20                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
21                    + x[i,j]) == 0)
22
23    # Indicator function
24    for i in range(n):
25        for j in range(n):
26            m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
27            m.add_constraint(-z[i,j] + A_G[i][j] <= 0)
28
29    # Product
30    for i in range(n):
31        for j in range(n):
32            for k in range(n):
33                m.add_constraint(y[i,k,j] - x[i,k] <= 0)
34                m.add_constraint(y[i,k,j] - x[k,j] <= 0)
35                m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)
36
37    # Symmetry
38    for i in range(n):
39        for j in range(n):
40            m.add_constraint(x[i,j] == x[j,i])
41
42    # Diagonal
43    for i in range(n):
44        m.add_constraint(x[i,i] == 0)
45
46    # Variables
47    for i in range(n):
48        for j in range(n):
49            m.add_constraint(x[i,j] - 1 <= 0)
50            for k in range(n):
51                m.add_constraint(y[i,k,j] - 1 <= 0)

```

```
51 return m, x
```

Listing 2: Linear Program.

```
1 def linear_program(G):
2     m = MixedIntegerLinearProgram(solver = 'GLPK', maximization=False)
3     m.solver_parameter("simplex_or_intopt", "intopt_only")
4
5     x = m.new_variable(real=True, nonnegative=True)
6     y = m.new_variable(real=True, nonnegative=True)
7     z = m.new_variable(real=True, nonnegative=True)
8
9     A_G = G.adjacency_matrix()
10    n = A_G.nrows()
11    M = n**2
12
13    # Indicator argument
14    for i in range(n):
15        for j in range(n):
16            if i == j:
17                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
18                    - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
19            else:
20                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
21                    + x[i,j]) == 0)
22
23    # Indicator function
24    for i in range(n):
25        for j in range(n):
26            m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
27            m.add_constraint(-z[i,j] + A_G[i][j] <= 0)
28
29    # Product
30    for i in range(n):
31        for j in range(n):
32            for k in range(n):
33                m.add_constraint(y[i,k,j] - x[i,k] <= 0)
34                m.add_constraint(y[i,k,j] - x[k,j] <= 0)
35                m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)
36
37    # Symmetry
38    for i in range(n):
39        for j in range(n):
40            m.add_constraint(x[i,j] == x[j,i])
41
42    # Diagonal
43    for i in range(n):
44        m.add_constraint(x[i,i] == 0)
45
46    # Variables
47    for i in range(n):
48        for j in range(n):
49            m.add_constraint(x[i,j] - 1 <= 0)
50            for k in range(n):
```

```

49         m.add_constraint(y[i,k,j] - 1 <= 0)
50     return m, x

```

Listing 3: Max Linear Program.

```

1  def max_linear_program(G):
2      m = MixedIntegerLinearProgram(solver = 'GLPK', maximization=False)
3      m.solver_parameter("simplex_or_intopt", "intopt_only")
4
5      x = m.new_variable(real=True, nonnegative=True)
6      y = m.new_variable(real=True, nonnegative=True)
7      z = m.new_variable(real=True, nonnegative=True)
8
9      A_G = G.adjacency_matrix()
10     n = A_G.nrows()
11     M = n**2
12
13     # Indicator argument
14     for i in range(n):
15         for j in range(n):
16             if i == j:
17                 m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
18                     - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
19             else:
20                 m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
21                     + x[i,j]) == 0)
22
23     # Indicator function
24     for i in range(n):
25         for j in range(n):
26             m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
27             m.add_constraint(-z[i,j] + A_G[i][j] <= 0)
28
29     # Product
30     for i in range(n):
31         for j in range(n):
32             for k in range(n):
33                 m.add_constraint(y[i,k,j] - x[i,k] <= 0)
34                 m.add_constraint(y[i,k,j] - x[k,j] <= 0)
35                 m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)
36
37     # Symmetry
38     for i in range(n):
39         for j in range(n):
40             m.add_constraint(x[i,j] == x[j,i])
41
42     # Diagonal
43     for i in range(n):
44         m.add_constraint(x[i,i] == 0)
45
46     # Variables
47     for i in range(n):
48         for j in range(n):
49             m.add_constraint(x[i,j] - 1 <= 0)

```

```

48         for k in range(n):
49             m.add_constraint(y[i,k,j] - 1 <= 0)
50
51     m.set_objective(- m.sum(x[i,j] for i in range(n) for j in range(n)))
52     return m, x

```

Listing 4: Min Linear Program.

```

1 def min_linear_program(G):
2     m = MixedIntegerLinearProgram(solver = 'GLPK', maximization=False)
3     m.solver_parameter("simplex_or_intopt", "intopt_only")
4
5     x = m.new_variable(real=True, nonnegative=True)
6     y = m.new_variable(real=True, nonnegative=True)
7     z = m.new_variable(real=True, nonnegative=True)
8
9     A_G = G.adjacency_matrix()
10    n = A_G.nrows()
11    M = n**2
12
13    # Indicator argument
14    for i in range(n):
15        for j in range(n):
16            if i == j:
17                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
18                    - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
19            else:
20                m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
21                    + x[i,j]) == 0)
22
23    # Indicator function
24    for i in range(n):
25        for j in range(n):
26            m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
27            m.add_constraint(-z[i,j] + A_G[i][j] <= 0)
28
29    # Product
30    for i in range(n):
31        for j in range(n):
32            for k in range(n):
33                m.add_constraint(y[i,k,j] - x[i,k] <= 0)
34                m.add_constraint(y[i,k,j] - x[k,j] <= 0)
35                m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)
36
37    # Symmetry
38    for i in range(n):
39        for j in range(n):
40            m.add_constraint(x[i,j] == x[j,i])
41
42    # Diagonal
43    for i in range(n):
44        m.add_constraint(x[i,i] == 0)
45
46    # Variables

```

```

45     for i in range(n):
46         for j in range(n):
47             m.add_constraint(x[i,j] - 1 <= 0)
48             for k in range(n):
49                 m.add_constraint(y[i,k,j] - 1 <= 0)
50
51     m.set_objective(m.sum(x[i,j] for i in range(n) for j in range(n)))
52     return m, x

```

Listing 5: Uniclqual Vertex Linear Program.

```

1  def find_leaf_stem(G):
2      n = G.order()
3      N = {v: set(G.neighbors(v)) | {v} for v in range(n)}
4      signature = {v: frozenset(N[v]) for v in range(n)}
5
6      contained = [[False] * n for _ in range(n)]
7      proper = [[False] * n for _ in range(n)]
8
9      for u in range(n):
10         for v in range(n):
11             if N[u] <= N[v]:
12                 contained[u][v] = True
13                 proper[u][v] = N[u] != N[v]
14
15     for s in range(n):
16         for u in N[s] - {s}:
17             if not proper[s][u]:
18                 continue
19             for v in N[u] - {u}:
20                 if not contained[u][v]:
21                     continue
22                 leaves = N[s] - {u, v}
23                 if all(signature[w] == signature[s] for w in leaves):
24                     return {"leaf": s, "stem": u, "vertex": v, "leaves":
25                             leaves}
26
27     return None
28
29 def uniclqual_vertex_linear_program(G):
30     m = MixedIntegerLinearProgram(solver = 'GLPK', maximization=False)
31     m.solver_parameter("simplex_or_intopt", "intopt_only")
32
33     x = m.new_variable(real=True, nonnegative=True)
34     y = m.new_variable(real=True, nonnegative=True)
35     z = m.new_variable(real=True, nonnegative=True)
36
37     A_G = G.adjacency_matrix()
38     n = A_G.nrows()
39     M = n**2
40
41     # Indicator argument
42     for i in range(n):
43         for j in range(n):

```

```

43         if i == j:
44             m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
45                                     - sum(x[i,k] for k in range(n)) + x[i,j]) == 0)
46         else:
47             m.add_constraint(z[i,j] - (sum(y[i,k,j] for k in range(n))
48                                     + x[i,j]) == 0)
49
50     # Indicator function
51     for i in range(n):
52         for j in range(n):
53             m.add_constraint(z[i,j] - M * A_G[i][j] <= 0)
54             m.add_constraint(-z[i,j] + A_G[i][j] <= 0)
55
56     # Product
57     for i in range(n):
58         for j in range(n):
59             for k in range(n):
60                 m.add_constraint(y[i,k,j] - x[i,k] <= 0)
61                 m.add_constraint(y[i,k,j] - x[k,j] <= 0)
62                 m.add_constraint(- y[i,k,j] + x[i,k] + x[k,j] - 1 <= 0)
63
64     # Symmetry
65     for i in range(n):
66         for j in range(n):
67             m.add_constraint(x[i,j] == x[j,i])
68
69     # Diagonal
70     for i in range(n):
71         m.add_constraint(x[i,i] == 0)
72
73     # Variables
74     for i in range(n):
75         for j in range(n):
76             m.add_constraint(x[i,j] - 1 <= 0)
77             for k in range(n):
78                 m.add_constraint(y[i,k,j] - 1 <= 0)
79
80     m.set_objective(-m .sum(x[i, j] for i in range(n) for j in range(n)))
81
82     # Find a leaf-stem structure using the largest closed neighbourhood
83     # clique
84     structure = find_leaf_stem(G)
85
86     if structure is None:
87
88         return m, x
89
90     s = structure["leaf"]
91     u = structure["stem"]
92
93     m.add_constraint(x[s, u] == 1)
94     m.add_constraint(x[u, s] == 1)
95
96     return m, x

```

Listing 6: Vertex Deletion Linear Program.

```

1 def vertex_deletion_linear_program(G):
2     fixed_edges = set()
3     m, x = unicliqual_vertex_linear_program(G)
4     for u, v in fixed_edges:
5         m.add_constraint(x[u, v] == 1)
6         m.add_constraint(x[v, u] == 1)
7     m.solve()
8
9     n = G.order()
10    solution_x = m.get_values(x)
11    integral_edges = {tuple(sorted((i, j))) for i in range(n) for j in
12                      range(i + 1, n) if abs(solution_x[i, j] - 1) <= 1e-6}
13    fixed_edges.update(integral_edges)
14    is_integral = not any(abs(solution_x[i, j] - round(solution_x[i, j]))
15                          > 1e-6 for i in range(n) for j in range(n))
16    if is_integral:
17        return m, x
18
19    unfixed_degree = {v: sum(tuple(sorted((u, v))) not in fixed_edges for
20                           u in G.neighbors(v)) for v in G}
21    v = min(unfixed_degree, key=lambda u: unfixed_degree[u])
22    remaining_vertices = [u for u in G if u != v]
23    G_minus = G.subgraph(remaining_vertices)
24    relabel_map = {u: i for i, u in enumerate(remaining_vertices)}
25    G_minus.relabel(relabel_map)
26    fixed_edges_minus = {tuple(sorted((relabel_map[u], relabel_map[w])))
27                        for u, w in fixed_edges if u in relabel_map and w in relabel_map}
28
29    H_minus = None
30    try:
31        m_minus, x_minus = unicliqual_vertex_linear_program(G_minus)
32        for u, w in fixed_edges_minus:
33            m_minus.add_constraint(x_minus[u, w] == 1)
34            m_minus.add_constraint(x_minus[w, u] == 1)
35        m_minus.solve()
36        n_minus = G_minus.order()
37        solution_x_minus = m_minus.get_values(x_minus)
38        is_integral_minus = not any(abs(solution_x_minus[i, j] - round(
39            solution_x_minus[i, j])) > 1e-6 for i in range(n_minus) for j
40            in range(n_minus))
41        if is_integral_minus:
42            H_minus = Graph(matrix([[round(solution_x_minus[i, j]) for j
43                in range(n_minus)] for i in range(n_minus)]))
44    except MIPSolverException:
45        pass
46
47    if H_minus is not None:
48        m_test, x_test = unicliqual_vertex_linear_program(G)
49        for u, w in fixed_edges:
50            m_test.add_constraint(x_test[u, w] == 1)
51            m_test.add_constraint(x_test[w, u] == 1)

```

```

45     for i, a in enumerate(remaining_vertices):
46         for j, b in enumerate(remaining_vertices):
47             if a != b and H_minus.has_edge(i, j):
48                 m_test.add_constraint(x_test[a, b] == 1)
49     try:
50         m_test.solve()
51         solution_x_test = m_test.get_values(x_test)
52         new_fixed_edges = {tuple(sorted((i, j))) for i in range(n) for
53                             j in range(i + 1, n) if abs(solution_x_test[i, j] - 1) <=
54                             1e-6}
55         added_edges = new_fixed_edges - fixed_edges
56         fixed_edges.update(new_fixed_edges)
57         is_integral_test = not any(abs(solution_x_test[i, j] - round(
58             solution_x_test[i, j])) > 1e-6 for i in range(n) for j in
59             range(n))
60         if is_integral_test:
61             return m_test, x_test
62         if added_edges:
63             m, x, solution_x = m_test, x_test, solution_x_test
64             return vertex_deletion_linear_program(G)
65     except MIPSolverException:
66         pass
67
68     return m, x

```

Listing 7: Recursive Vertex Deletion Linear Program.

```

1 def vertex_deletion_recursive_linear_program(G):
2     fixed_edges = set()
3     m, x = unicliqual_vertex_linear_program(G)
4     m.solve()
5
6     n = G.order()
7     solution_x = m.get_values(x)
8     integral_edges = {tuple(sorted((i, j))) for i in range(n) for j in
9                       range(i + 1, n) if abs(solution_x[i, j] - 1) <= 1e-6}
10    fixed_edges.update(integral_edges)
11    is_integral = not any(abs(solution_x[i, j] - round(solution_x[i, j]))
12                          > 1e-6 for i in range(n) for j in range(n))
13    if is_integral:
14        return m, x
15
16    unfixed_degree = {v: sum(tuple(sorted((u, v))) not in fixed_edges for
17                             u in G.neighbors(v)) for v in G}
18    v = min(unfixed_degree, key=lambda u: unfixed_degree[u])
19    remaining_vertices = [u for u in G if u != v]
20    G_minus = G.subgraph(remaining_vertices)
21    relabel_map = {u: i for i, u in enumerate(remaining_vertices)}
22    G_minus.relabel(relabel_map)
23    fixed_edges_minus = {tuple(sorted((relabel_map[u], relabel_map[w])))
24                        for u, w in fixed_edges if u in relabel_map and w in relabel_map}
25
26    H_minus = None
27    try:

```

```

24     m_minus, x_minus = unicliqual_vertex_linear_program(G_minus)
25     for u, w in fixed_edges_minus:
26         m_minus.add_constraint(x_minus[u, w] == 1)
27         m_minus.add_constraint(x_minus[w, u] == 1)
28     m_minus.solve()
29     n_minus = G_minus.order()
30     solution_x_minus = m_minus.get_values(x_minus)
31     is_integral_minus = not any(abs(solution_x_minus[i, j] - round(
32         solution_x_minus[i, j])) > 1e-6 for i in range(n_minus) for j
33         in range(n_minus))
34     if is_integral_minus:
35         H_minus = Graph(matrix([[round(solution_x_minus[i, j]) for j
36             in range(n_minus)] for i in range(n_minus)]))
37     else:
38         m_recursive, x_recursive =
39             vertex_deletion_recursive_linear_program(G_minus)
40         solution_x_minus = m_recursive.get_values(x_recursive)
41         is_integral_recursive = not any(abs(solution_x_minus[i, j] -
42             round(solution_x_minus[i, j])) > 1e-6 for i in range(
43             n_minus) for j in range(n_minus))
44         if is_integral_recursive:
45             H_minus = Graph(matrix([[round(solution_x_minus[i, j]) for
46                 j in range(n_minus)] for i in range(n_minus)]))
47     except MIPSolverException:
48         pass
49
50     if H_minus is not None:
51         m_test, x_test = unicliqual_vertex_linear_program(G)
52         for u, w in fixed_edges:
53             m_test.add_constraint(x_test[u, w] == 1)
54             m_test.add_constraint(x_test[w, u] == 1)
55         for i, a in enumerate(remaining_vertices):
56             for j, b in enumerate(remaining_vertices):
57                 if a != b and H_minus.has_edge(i, j):
58                     m_test.add_constraint(x_test[a, b] == 1)
59         try:
60             m_test.solve()
61             solution_x_test = m_test.get_values(x_test)
62             new_fixed_edges = {tuple(sorted((i, j))) for i in range(n) for
63                 j in range(i + 1, n) if abs(solution_x_test[i, j] - 1) <=
64                 1e-6}
65             added_edges = new_fixed_edges - fixed_edges
66             fixed_edges.update(new_fixed_edges)
67             is_integral_test = not any(abs(solution_x_test[i, j] - round(
68                 solution_x_test[i, j])) > 1e-6 for i in range(n) for j in
69                 range(n))
70             if is_integral_test:
71                 return m_test, x_test
72             if added_edges:
73                 m, x, solution_x = m_test, x_test, solution_x_test
74         except MIPSolverException:
75             pass
76
77     return m, x

```

Listing 8: Uniclqual Rounding Linear Program.

```

1 def integer_components_and_ranked_vertex_edges(x_val, n, eps=1e-6):
2     G_int = Graph()
3     G_int.add_vertices(range(n))
4
5     for (i, j), v in x_val.items():
6         if i < j and v > 1 - eps:
7             G_int.add_edge(i, j)
8
9     components = G_int.connected_components()
10
11     comp_index = {}
12     for idx, C in enumerate(components):
13         for v in C:
14             comp_index[v] = idx
15
16     vertex_score = {}
17     for (i, j), v in x_val.items():
18         if i >= j:
19             continue
20         if eps < v < 1 - eps:
21             w = abs(v - 0.5)
22             vertex_score[i] = vertex_score.get(i, 0) + w
23             vertex_score[j] = vertex_score.get(j, 0) + w
24
25     if not vertex_score:
26         return components, comp_index, None, []
27
28     v_star = max(vertex_score.items(), key=lambda t: (t[1], -t[0]))[0]
29
30     incident_edges = []
31     for (i, j), v in x_val.items():
32         if eps < v < 1 - eps:
33             if i == v_star or j == v_star:
34                 a, b = min(i, j), max(i, j)
35                 incident_edges.append((a, b, v, abs(v - 0.5)))
36
37     incident_edges.sort(key=lambda t: (-t[3], t[0], t[1]))
38
39     return components, comp_index, v_star, incident_edges
40
41 def unicliqual_rounding_linear_program(G, threshold=0.5):
42     fixed_assignments = []
43     n = G.order()
44
45     m, x = unicliqual_vertex_linear_program(G)
46     m.solve()
47
48     for step in range(n * (n - 1) // 2):
49         x_val = m.get_values(x)
50         _, comp_index, v_star, edges =
            integer_components_and_ranked_vertex_edges(x_val, n)

```

```

51
52     if v_star is None:
53         return m, x
54
55     i, j, v, _ = edges[0]
56     fix_val = 1 if comp_index[i] != comp_index[j] else (1 if v >
57         threshold else 0)
58
59     m_try, x_try = unicliqual_vertex_linear_program(G)
60     for a, b, val in fixed_assignments:
61         m_try.add_constraint(x_try[a,b] == val)
62         m_try.add_constraint(x_try[b,a] == val)
63     m_try.add_constraint(x_try[i,j] == fix_val)
64     m_try.add_constraint(x_try[j,i] == fix_val)
65
66     try:
67         m_try.solve()
68     except MIPSolverException:
69         return m, x
70
71     fixed_assignments.append((i, j, fix_val))
72     m, x = m_try, x_try
73
74     return m, x

```

Bibliography

- [BK73] Coen Bron and Joep Kerbosch. “Algorithm 457: Finding all cliques of an undirected graph”. In: *Communications of the ACM* 16.9 (1973), pp. 575–577.
- [BMR14] Stephan Brandt, Janina Muttel, and Dieter Rautenbach. “The circumference of the square of a connected graph”. In: *Combinatorica* 34.5 (2014), pp. 547–559.
- [Coc+13] Manfred Cochefert, Jean-François Couturier, Petr A. Golovach, Dieter Kratsch, and Daniël Paulusma. “Sparse Square Roots”. In: *Lecture Notes in Computer Science* 8165 (2013), pp. 177–188.
- [Duf25] Christopher Duffy. “Quasi-Transitive Mixed Graphs and Undirected Squares of Oriented Graphs”. In: *Australasian Journal of Combinatorics* 92.2 (2025), pp. 142–163.
- [Far+09] Babak Farzad, Lap Chi Lau, Van Bang Le, and Nguyen Ngoc Tuy. “Computing Graph Roots Without Short Cycles”. In: *Proceedings of the 26th International Symposium on Theoretical Aspects of Computer Science (STACS 2009)*. Vol. 3. Leibniz International Proceedings in Informatics (LIPIcs). 2009, pp. 397–408.
- [FBB23] Leon Festinger, Françoise Bahoken, and Laurent Beauguitte. “Leon Festinger, 1949, The Analysis of Sociograms Using Matrix Algebra. Version bilingue et commentée”. In: (2023).
- [FK12] Babak Farzad and Majid Karimi. “Square-Root Finding Problem In Graphs, A Complete Dichotomy Theorem”. In: (2012).
- [Fle74] Herbert Fleischner. “The square of every nonseparable graph is Hamiltonian”. In: *Journal of Combinatorial Theory, Series B* 16.1 (1974), pp. 29–34.
- [Gel70] Dennis P. Geller. “The Square Root of a Digraph”. In: *Journal of Combinatorial Theory* 9.1 (1970), pp. 65–68.
- [HKT67] Frank Harary, Richard M. Karp, and William T. Tutte. “A criterion for planarity of the square of a graph”. In: *Journal of Combinatorial Theory* 2 (1967), pp. 406–410.
- [Kön36] Dénes König. *Theorie der endlichen und unendlichen Graphen*. Akademische Verlagsgesellschaft, 1936.
- [Lau06] Lap Chi Lau. “Bipartite Roots of Graphs”. In: *ACM Transactions on Algorithms* 2.2 (2006), pp. 178–208.
- [LC04] Lap Chi Lau and Derek G. Corneil. “Recognizing Powers of Proper Interval, Split, and Chordal Graphs”. In: *SIAM Journal on Discrete Mathematics* 18.1 (2004), pp. 83–102.
- [LS95] Yaw-Ling Lin and Steven S. Skiena. “Algorithms for square roots of graphs”. In: *SIAM Journal on Discrete Mathematics* 8.1 (1995), pp. 99–118.
- [LT07] Van Bang Le and Nguyen Ngoc Tuy. “The square of a block graph”. In: *Discrete Mathematics* 307.21 (2007), pp. 2706–2715.

- [MS94] Rajeev Motwani and Madhu Sudan. “Computing roots of graphs is hard”. In: *Discrete Applied Mathematics* 54.1 (1994), pp. 81–88.
- [Muk67] A. Mukhopadhyay. “The Square Root of a Graph”. In: *Journal of Combinatorial Theory* 2 (1967), pp. 290–295.
- [PA23] Or Peres and C. Avin. “Distributed Demand-aware Network Design using Bounded Square Root of Graphs”. In: *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications* (2023), pp. 1–10.
- [RH60] Ian C. Ross and Frank Harary. “The square of a tree”. In: *The Bell System Technical Journal* 39.3 (1960), pp. 641–647.
- [Sch78] Thomas J. Schaefer. “The complexity of satisfiability problems”. In: *Proceedings of the Tenth Annual ACM Symposium on Theory of Computing* (1978), pp. 216–226.
- [Zho+25] Bo Zhou, Siyang Liu, Lei Chen, and Qi Dai. “Graph-Root: Prediction of Root-Associated Proteins in Maize, Sorghum, And Soybean Based on Graph Convolutional Network and Network Embedding Method”. In: *Current Bioinformatics* (2025), pp. 828–839.